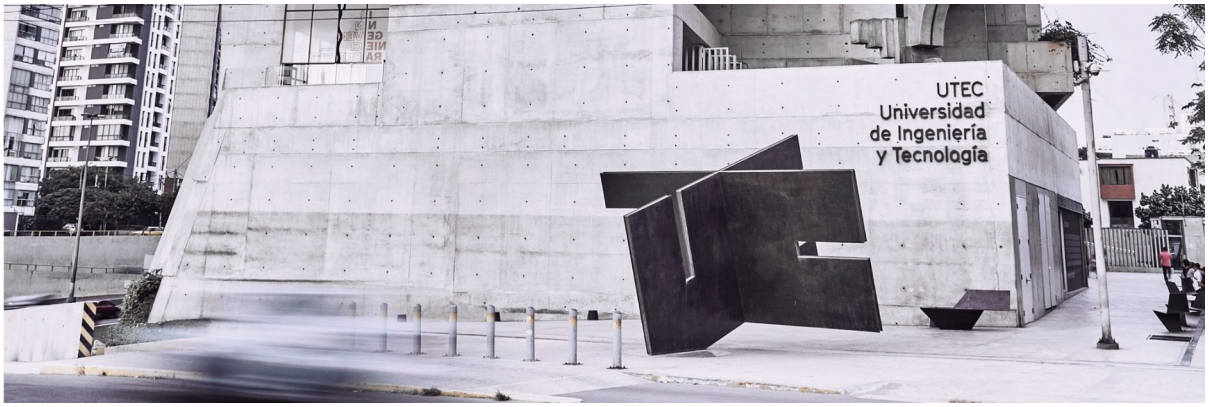


Informe de Pruebas de Rendimiento

Grupo: Trabajo individual: Johar Jared Barzola Estrella (G4)
Fecha: 12/11/2026 (corridas del 11/11/2026, 18:29–21:28, hora de Lima)

Aplicación bajo prueba: Qolqe Wallet 1.0.0, API REST (qolqe/ sin cambios)
Herramienta: k6 2.3.0, con Qolqe en Docker (--cpus=1 --memory=1g)

---

1. Alcance y justificación

Contexto de negocio (2-3 líneas):

Qolqe Wallet es una billetera digital: saldo, transferencias entre usuarios, recargas de celular y pago de servicios. Antes de la campaña “Quincena Qolqe” (días 15 y último de cada mes, de 12:00 a 12:15, con el triple de usuarios simultáneos) el negocio necesita saber si cumple en la hora punta (3,000 sesiones por hora), si cumple en el pico de la campaña y hasta cuántos usuarios simultáneos puede crecer antes de dejar de cumplir.

Flujos críticos y criterio de selección:

Criterios del enunciado: **frecuencia** (porcentaje de la tabla del requerimiento), **impacto en el dinero del usuario** y **riesgo** (técnico, con evidencia en el código, y de negocio). Un flujo es crítico si cumple al menos dos de los tres: frecuencia ≥ 15 %, mueve dinero, o riesgo alto con evidencia. El inicio de sesión entra aunque no mueva dinero, porque todas las sesiones pasan por él. Las peticiones de cada flujo siguen el orden de la pantalla (`qolqe/web/static/app.js`).

ID	Flujo	Peticiones	Frecuencia	Dinero	Riesgo (evidencia)
F-01	Iniciar sesión	`POST /api/auth/login`	1 por sesión (0.83/s en hora punta; 2.5/s en campaña)	Indirecto	**Alto.** PBKDF2-SHA256 de 200,000 iteraciones por login (`credenciales.py:13,24`). En la campaña “la mayoría inicia sesión en los primeros 5 minutos”: la ráfaga cae sobre la operación más cara. Bloqueo tras 3 PIN fallidos (RN-04)
F-02	Transferir a otro usuario Qolqe	`GET …/comision` → `POST /api/transferencias`	15 %	**Directo**	**Alto.** El antifraude duerme 40–120 ms dentro de la petición y ocupa un hilo (`operaciones.py:99-102`). Ventana de carrera entre validar y escribir (DEF-21). “Ninguna… a medias ni aplicada dos veces”
F-03	Consultar saldo y perfil	`GET /api/cuentas/me/saldo` + `GET /api/cuentas/me`	**30 %** (la más frecuente)	Indirecto	Medio: 3 consultas con índice por petición autenticada
F-04	Recargar un celular	`GET …/configuracion` → `POST /api/recargas`	10 %	**Directo**	Medio: recorre `recargas` sin índice en cada recarga; la tabla crece en la resistencia

ID	Flujo	Peticiones	Frecuencia	Dinero	Riesgo (evidencia)
F-05	Pagar servicios	`GET …/empresas` → `GET …/deudas` → `POST /api/servicios/pagos`	4 % (+ 6 % solo consulta)	**Directo**	Medio-alto: las deudas no tienen dueño (`servicios.py:105`); un pago repetido da 409. Es el dinero con menos muestras
F-06	Consultar movimientos	`GET /api/movimientos?pagina=1` (30 días)	15 %	No	**Alto técnico.** Consulta N+1: un `SUM` por fila del rango (`movimientos.py:113-123`), ≈ 243 consultas por petición con 1,000 movimientos en 120 días
—	Ver y buscar contactos	`GET /api/contactos` (+ `?buscar=`)	10 %	No	Bajo. Carga de fondo
—	Otras (límites, alertas, solicitudes, dispositivos, sesiones)	5 × `GET`, 2 % cada uno	10 %	No	Bajo. Carga de fondo

Todos los flujos, críticos o no, están en la mezcla, porque el perfil operacional tiene que estar completo. Los críticos tienen umbral propio por endpoint y se analizan primero.

Capa objetivo:

- ☐ Web
- ☒ API
- ☐ Ambas

Justificación:

- Los umbrales hablan del tiempo en que “responde” la operación, y eso lo determina el servidor.
- La web es HTML estático más `static/app.js`, que llama a los mismos endpoints con `fetch`. El script reproduce la secuencia exacta de peticiones de cada pantalla: la carga sobre la API es la misma que genera la web.
- Simular 300 o más navegadores en una laptop mediría al navegador (Chromium) y no a Qolqe.
- Los archivos estáticos quedan fuera: se asume que el navegador o una CDN los tiene en caché (S-16).
- Es el razonamiento del laboratorio de S13: “¿Dónde probar? → Solo API → Habla de un endpoint, no de navegación web”.

---

2. Herramienta seleccionada

Se eligió **k6 2.3.0** (Grafana Labs, versión estable de Homebrew, publicada el 21-sep-2026) [11][13]. El requerimiento expresa la carga como **tasa de llegada** (3,000 sesiones por hora) y exige sostenerla “sin acumular retrasos”. k6 modela esa tasa con ejecutores de llegada (modelo abierto): cada iteración es una sesión y, si Qolqe se vuelve lento, k6 no reduce la carga, sino que lo registra en `dropped_iterations`, que se usa como umbral [1][3][10]. Los SLA se escriben como umbrales en el script (p95 < 800 ms en consultas e inicio de sesión, p95 < 1.5 s en dinero, error ≤ 1 %) y k6 termina con código 99 si alguno falla: es el paso 1 del análisis de S14, “¿pasó o falló?” [2][4]. Además da un dashboard en vivo y un reporte HTML, exporta cada muestra a CSV para cruzarla con el CPU y la memoria medidos con psutil y consume poco (binario en Go, sin JVM), algo clave porque corre en la misma laptop [7][8][9].

Criterios eliminatorios para este requerimiento (salen de los datos del caso):

Criterio	k6	JMeter	Gatling	Locust	Artillery	Taurus
C1 Modelo abierto (sesiones/h) y métrica de retraso acumulado con umbral ("sin acumular retrasos")	Sí	Parcial	Parcial	No	Sí	Parcial
C2 Umbrales del SLA por grupo de operación, con código de salida	Sí	No	Sí	Parcial	Sí	Sí
C3 Reporte HTML local y datos crudos por petición	Sí	Sí	Parcial	Parcial	No	Parcial
C4 Generador liviano en la misma laptop (CPU de la herramienta < ~80 %)	Sí	Parcial	Sí	Parcial	Parcial	No
Total "Sí"	**4**	1	2	0	2	1

Comparación detallada (versiones vigentes en noviembre de 2026):

Criterio	k6 2.3.0	JMeter 5.6.3	Gatling 3.16.0	Locust 2.47.0	Artillery 2.0.34	Taurus 1.17.2
Modelo abierto	Sí, nativo: `constant-arrival-rate`, `ramping-arrival-rate` [1] [10]	Parcial: Open Model Thread Group, experimental desde 5.5 [18]	Sí: `injectOpen` [23]	No: población fija de usuarios [30]	Sí: `phases` con `arrivalRate` [35]	Parcial: `concurrency` con tope de `throughput` [42]
Retraso acumulado con umbral	Sí: `dropped_iterations` [3]	No nativo	Parcial: gráfico de usuarios activos, sin aserción [24]	No	Parcial: `vusers.skipped`, solo con `maxVusers` [39]	No
Umbrales como código y código de salida	Sí: `thresholds` por etiqueta, código 99, `abortOnFail` [2] [4]	No: sale con 0 aun con fallas [21]	Sí: `assertions` [24]	Parcial: hook en Python [31][33]	Sí: plugin `ensure` [37]	Sí: `passfail` [41]
Mezcla ponderada por operación	Parcial: función propia de ~10 líneas	Sí: Throughput Controller [17]	Sí: `randomSwitch` [28]	Sí: `@task(peso)` [30]	Parcial: pesos por escenario [35]	Hereda del ejecutor
Think time de 10–25 s	Sí: `sleep()` con cualquier distribución	Sí: Uniform Random Timer [17]	Sí: `pause(10, 25)` [28]	Sí: `between(10, 25)` [30]	Parcial: `think` con `jitter` [36]	Sí
Usuario distinto por VU desde CSV	Sí: `SharedArray` + `vu.idInTest` [6]	Sí: CSV Data Set Config [17]	Sí: feeder `csv().queue()` [29]	Parcial: código propio	Parcial: `payload` en secuencia [35]	Sí
Reporte HTML / en vivo	Sí / Sí [7]	Sí / No: dashboard `-e -o` al final [19]	Sí / No [26]	Sí / Sí [31]	No / No: el HTML local se eliminó [38]	Parcial: el web es de BlazeMeter
Datos crudos por petición	Sí: `--out csv` con marca de tiempo [8]	Sí: `.jtl`	No [27]	Parcial: CSV agregado [31]	Parcial: JSON agregado	Depende del ejecutor
Huella del generador	Baja: binario en Go, 1–5 MB por VU [9]	Alta: JVM, heap de 1 GB, un hilo por VU [20]	Media: JVM asíncrona	Media: 1 núcleo por proceso [32]	Media: Node.js	Alta: bzt + la herramienta
Curva de aprendizaje	Baja: JavaScript/TypeScript [12]; el repo ya usa TypeScript en `tests/e2e`	Baja en la GUI (laboratorio); alta para la lógica de sesión	Media-alta: DSL Java/Kotlin/Scala, JDK y Maven [25]	Baja: Python (el repo usa pytest)	Baja-media: YAML + JS	Media: YAML + la herramienta
Demo en vivo	Muy buena: un comando y dashboard en `localhost:5665` [7]	Media: el gráfico llega al final	Media: hay que compilar	Muy buena: UI web	Media	Media

Ventajas de k6 para este caso:

- **El modelo de carga es el texto del requerimiento:** `rate: 50, timeUnit: '1m'` es literalmente 3,000 sesiones/h (M-01). La campaña y el estrés son etapas de tasa de llegada (`ramping-arrival-rate`).
- **"Sin acumular retrasos" se puede verificar:** con modelo abierto el retraso no se esconde. En un modelo cerrado (Thread Group de JMeter, usuarios fijos de Locust)

el generador espera cuando Qolqe se pone lento, llegan menos peticiones y el retraso queda oculto.

- **Umbral = SLA** con métrica + percentil + valor + carga (S14), por grupo (`consulta`, `dinero`) y por endpoint, filtrado por la fase medida. El código de salida es el veredicto.
- **Los 4xx de regla de negocio se separan** de los errores de rendimiento con `expectedStatuses` y contadores propios, como pide el enunciado [16].
- **Datos por VU:** `SharedArray` + `exec.vu.idInTest` garantiza que dos sesiones simultáneas nunca compartan usuario [6].
- **Evidencia para `resultados/`:** reporte HTML autocontenido, crudos en CSV con marca de tiempo y resumen JSON.

Limitaciones de k6:

- No tiene GUI ni grabador. No hace falta: se prueba la API y las peticiones salen de `app.js`.
- No tiene controlador nativo de mezcla ponderada: se programó una función con la tabla del requerimiento.
- No mide CPU ni memoria del servidor: se midieron aparte con psutil y `docker stats` (igual con cualquier herramienta).
- Da por cumplido un umbral sin muestras; el resumen propio (`k6/resumen.js`) lo marca como “sin muestras”.
- Su `rate` de peticiones divide por toda la corrida, no por la fase: el throughput mínimo (U-04) se verifica como conteo en la fase estable.

Por qué no las otras:

- **JMeter** (visto en el laboratorio): el modelo abierto es experimental, no mide sesiones descartadas y no tiene umbrales con código de salida (sale con 0 aun con fallas) [21]. JVM de 1 GB y sin versión nueva desde ene-2024 [22]. Su fuerte es el dashboard HTML.
- **Gatling:** la alternativa más sólida en modelo de carga y aserciones, pero requiere JDK y Maven/Gradle, no exporta datos crudos [27] y la versión libre no tiene vista en vivo [26].
- **Locust:** encaja con el Python del repo y su UI es buena para una demo, pero no tiene modelo abierto, que es justo lo que pide “sin acumular retrasos”.
- **Artillery:** el modelo abierto calza, pero ya no genera HTML local [38] y los pesos son por escenario, no por operación.
- **Taurus:** agrega una capa sobre otra herramienta y su modelo es cerrado con tope de RPS.

Equivalencias con el laboratorio de JMeter (S13):

JMeter (laboratorio)	k6 (este proyecto)
Thread Group (Threads, Ramp-up, Loop)	`scenarios` con ejecutor de llegada (`constant-arrival-rate`, `ramping-arrival-rate`)
HTTP Request + HTTP Header Manager	`http.get()` / `http.post()` con `Authorization: Bearer …`
CSV Data Set Config	`SharedArray` + `exec.vu.idInTest` (`datos/usuarios.csv`)
Constant / Uniform Random Timer	`sleep(pausa())`, triangular(10, 19, 25) s (S-14)
Throughput Controller (%)	Función de elección ponderada con la tabla del requerimiento
Transaction Controller	Etiquetas `op` y `grupo`
Response Assertion	Validación del cuerpo en cada respuesta
Summary Report / `-e -o reporte/`	Resumen final (p50, p90, p95, p99) / `reporte.html`

JMeter (laboratorio)	k6 (este proyecto)
`-l resultados.jtl`	`--out csv=muestras.csv.gz`
(no existe)	`thresholds` con código de salida 99

No se corrió un contraste con JMeter (era opcional en el plan): la comparación de esta sección es documental, con las fuentes citadas.

Fuentes: [1] grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed · [2] …/using-k6/thresholds · [3] …/scenarios/concepts/dropped-iterations · [4] grafana.com/docs/learning-paths/automate-k6-cicd/handle-exit-code · [6] …/examples/data-parameterization · [7] …/results-output/web-dashboard · [8] …/results-output/real-time/csv · [9] …/testing-guides/running-large-tests · [10] …/executors/ramping-arrival-rate · [11] github.com/grafana/k6/releases · [12] grafana.com/blog/2025/05/07/grafana-k6-1.0-release · [13] formulae.brew.sh/formula/k6 · [16] …/k6-http/set-response-callback · [17] jmeter.apache.org/usermanual/component_reference.html · [18] blazemeter.com/blog/open-model-thread-group · [19] jmeter.apache.org/usermanual/generating-dashboard.html · [20] jmeter.apache.org/usermanual/best-practices.html · [21] blazemeter.com/blog/jmeter-github-testing · [22] jmeter.apache.org/changes.html · [23] docs.gatling.io/concepts/injection · [24] docs.gatling.io/concepts/assertions · [25] docs.gatling.io/reference/deploy/install-local · [26] docs.gatling.io/release-notes/gatling/upgrading/3.11-to-3.12 · [27] community.gatling.io/t/…/9736 · [28] docs.gatling.io/concepts/scenario · [29] docs.gatling.io/concepts/session/feeders · [30] docs.locust.io/en/stable/writing-a-locustfile.html · [31] docs.locust.io/en/stable/configuration.html · [32] docs.locust.io/en/stable/running-distributed.html · [33] docs.locust.io/en/latest/running-without-web-ui.html · [35] artillery.io/docs/reference/test-script · [36] artillery.io/docs/reference/engines/http · [37] artillery.io/docs/reference/extensions/ensure · [38] artillery.io/docs/reference/cli/report · [39] artillery.io/docs/reference/reported-metrics · [41] gettaurus.org/docs/PassFail · [42] gettaurus.org/docs/ExecutionSettings. Consultadas en noviembre de 2026. Curso: enunciado P2 Parte 3; S13 (Pruebas de rendimiento y laboratorio de JMeter); S14 (Pruebas de rendimiento II).

---

3. Tipos de prueba seleccionados

Regla de S13: “el tipo lo define la pregunta, no la cantidad de usuarios”. Se ejecutaron los cuatro tipos; cada uno responde a una frase del requerimiento:

Tipo	¿Aplica?	Justificación (conectar con el requerimiento)
Carga	☒ Sí / ☐ No	“Necesitamos saber si cumplimos en hora punta” y “sostener el volumen de la hora punta sin acumular retrasos”. Es la carga esperada: 3,000 sesiones/h = **100 usuarios simultáneos** (M-03). Caso A del ejercicio 1 de S13. Llenado de 120 s → **15 min de estado estable** → drenaje. Ejecutada: `carga/20261111-1833`
Estrés	☒ Sí / ☐ No	“Hasta cuántos usuarios simultáneos podemos crecer antes de dejar de cumplir” y “qué tan lejos está de su límite”. Caso D de S13: “¿se degrada con gracia o se cae?”. Rampa continua de 100 a L_top, más allá del límite estimado; da la **capacidad** (U-08). Ejecutada: `estres/20261111-1940` (y con 2 CPU, `escalabilidad/20261111-2103`)
Pico (spike)	☒ Sí / ☐ No	Campaña “Quincena Qolqe”: “el triple de usuarios simultáneos” de 12:00 a 12:15 y “la mayoría inicie sesión en los primeros 5 minutos”. Salto abrupto y concentrado (caso B de S13): 100 → ×3 en 30 s → **300** → 100, con recuperación e integridad del dinero (U-06, U-07). Ejecutada: `pico/20261111-1918`
Resistencia (soak)	☒ Sí / ☐ No	“Qolqe opera todos los días de 6:00 a 24:00 sin reinicios” (18 h). Caso C de S13: fugas de memoria, hilos o conexiones. Versión acotada de **60 min** a la carga de hora punta y proyección a las 54,000 sesiones de un día (S-11, S-12); la diferencia con 18 h se declara en §6. Ejecutada: `resistencia/20261111-1958`

Además se corrieron dos corridas de apoyo, que no son tipos de prueba: la **referencia de 1 VU** (línea base del entorno, V-06; `referencia/20261111-1828`) y la **calibración** (CPU por sesión para fijar la cima del estrés, M-14; `calibracion/20261111-1830`). La escalabilidad no está en la plantilla; se ejecutó como opcional (el mismo estrés con `--cpus=2`) porque sirve de evidencia del cuello de botella. También se ensayó la demo de clase (`demo/20261111-2125`).

4. Modelo de carga

Modelo **abierto** (por tasa de llegada): cada VU de k6 ejecuta una **sesión** completa (login + 6 operaciones + pausas). La carga se fija en sesiones por minuto y los usuarios simultáneos salen de la Ley de Little (ver “Origen de los valores”).

Parámetro	Tipo 1: Carga	Tipo 2: Estrés	Tipo 3: Pico	Tipo 4: Resistencia
VUs	**100** usuarios simultáneos (50 sesiones/min). k6 reserva 150 VUs, máximo 200 (M-15)	**100 → L_top = 2,400** (hipótesis; la calibración sugirió 1,100, ver M-14). Reserva 2,500, máximo 3,000	**100 → 300 → 100**. Reserva 400, máximo 600	**100** (reserva 150, máximo 200)
Ramp-up (s)	**120** (llenado natural = duración de la sesión, M-24)	180 de base a 100 + **1,380** de rampa (100 → 2,400: +100 simultáneos por minuto)	120 de llenado + **salto de 30** (50 → 150 sesiones/min); 300 simultáneos ≈ 120 s después	**120**
Duración (s)	**1,020** de llegadas (120 de llenado + **900 medidos**) + hasta 180 de drenaje	**1,680** planificados (180 + 1,380 + 120 de meseta). Real: k6 cortó a los **952 s** por la red de seguridad del 20 % (§6)	**1,140** de llegadas (240 de hora punta previa + **900 de campaña**, 12:00–12:15) + hasta 180 de drenaje	**3,720** de llegadas (120 + **3,600 medidos**) + hasta 180 de drenaje
Think time (ms)	10,000–25,000 entre acciones, triangular con moda 19,000 y **media 18,000** (6 pausas); más 12,000 de cierre sin peticiones	Ídem	Ídem	Ídem
Distribución de operaciones (%)	Login 1 por sesión; saldo y perfil 30; movimientos 15; transferir 15; contactos 10; recarga 10; consulta de deudas 6; pago 4; otras 10 (5 GET al 2 %)	Ídem	Ídem	Ídem

Origen de los valores (dato de negocio y cálculos):

Dos conceptos que el material del curso no trae (aporte propio, en palabras simples):

- **Modelo abierto y modelo cerrado.** En el abierto los usuarios llegan a un ritmo fijo (50 por minuto) aunque Qolqe esté lento; si Qolqe se atrasa, los usuarios se acumulan, y eso es lo que el requerimiento pide detectar. En el cerrado (Thread Group de JMeter) hay un número fijo de usuarios que repiten la sesión; si Qolqe se pone lento, esperan, llegan menos peticiones y el retraso se esconde.
- **Ley de Little.** Usuarios a la vez = usuarios que llegan por segundo × segundos que se queda cada uno (L = λ·W). Si a un restaurante llegan 10 personas por hora y cada una se queda media hora, hay 5 mesas ocupadas. Con esta ley se traducen “sesiones por hora” a “usuarios simultáneos” y al revés.

ID	Dato del requerimiento	Cálculo	Valor
M-01	“se inician unas 3,000 sesiones por hora”	3,000 ÷ 3,600 s	**λ = 0.833 sesiones/s = 50/min**

ID	Dato del requerimiento	Cálculo	Valor
M-02	“Una sesión dura en promedio 2 minutos”	—	**W = 120 s**
M-03	M-01 y M-02	L = λ·W = 0.833 × 120	**100 usuarios simultáneos** en hora punta
M-04	“pausa de 10 a 25 segundos (18 s en promedio)”	Login + 6 operaciones = 7 acciones → 6 pausas. Triangular(10, 19, 25): media (10 + 19 + 25) ÷ 3 = 18 s (S-14)	**6 × 18 = 108 s** de pausas por sesión
M-05	M-02 − M-04 (S-13)	120 − 108	**12 s de cierre** sin peticiones
M-06	Tabla de operaciones	6 operaciones por sesión, sorteadas con reemplazo con los pesos de la tabla	Mezcla del requerimiento
M-07	Pantallas de `app.js` (S-15, S-16)	Peticiones por operación	Login 1 · saldo y perfil 2 · movimientos 1 · transferir 2 · contactos 2 · recarga 2 · deudas 2 · pago 3 · otras 1
M-08	M-06 y M-07	Por operación: 0.30·2 + 0.15·1 + 0.15·2 + 0.10·2 + 0.10·2 + 0.06·2 + 0.04·3 + 0.10·1 = 1.79. Por sesión: 1 + 6 × 1.79 = 11.74	**9.78 req/s** en hora punta; dinero **1.45 POST/s**; logins **0.83/s**
M-09	“el triple de usuarios simultáneos… la mayoría inicie sesión en los primeros 5 minutos” (S-07, S-08)	L_pico = 3 × 100 = 300 → λ = 300 ÷ 2 min = 150/min de 12:00 a 12:05; luego 50/min hasta 12:15	**150 → 50 sesiones/min**; 1,250 sesiones, el **58 %** en los primeros 5 min
M-10	“la mayoría inicie sesión en los primeros 5 minutos”	La tasa se triplica en 30 s	**Salto de 30 s**
M-11	Hora punta previa al pico	120 s de llenado + 120 s medidos (fase `base`)	**4 min** antes de las 12:00
M-12	Hora punta 12:00–13:00	15 min estables = 750 sesiones; el endpoint de dinero más escaso tiene ≥ 100 muestras (pagos: 0.2/s × 900 s = 180)	**15 min estables**
M-13	“6:00 a 24:00 sin reinicios” (S-11, S-12)	60 min estables = 3,000 sesiones; proyección a 18 × 3,000	**60 min**, proyección a **54,000 sesiones**
M-14	“hasta cuántos usuarios simultáneos”	L_sat = W ÷ D (D = s de CPU por sesión, medido en la calibración); L_top = ⌈1.33 × L_sat⌉ a la centena. Hipótesis previa: D ≈ 69 ms → L_top = 2,400	Calibración (`calibracion/20261111-1830`): **D = 157 ms, L_sat = 763, L_top sugerido = 1,100**. Usado: **2,400** = máx(1,100; 2,400), porque con 1 VU la calibración sobrestima D (§6)
M-15	Técnico: techo del generador	preAllocatedVUs ≈ 1.5 × L y maxVUs ≈ 2 × L (estrés: L_top + 100 y 1.25 × L_top)	Carga y resistencia 150/200 · pico 400/600 · estrés 2,500/3,000
M-16	“un usuario distinto por VU”	Usuarios = maxVUs del escenario más grande (1.25 × 2,400)	**3,000 usuarios** (200 de la semilla + 2,800 creados por API)
M-24	Calentamiento	= W: con llegada constante, en 120 s se llena la concurrencia y se calienta la caché	**120 s**, fuera del veredicto

- **El ramp-up sale del dato, no se elige:** con llegada constante de 50/min los simultáneos suben solos de 0 a 100 en 120 s, lo que dura una sesión (M-02). Es la forma de “0 → 200 VUs en 2 min” de S13.
- **Duración de una sesión simulada:** de 72 s (6 × 10 + 12) a 162 s (6 × 25 + 12), con media de 120 s más los tiempos de respuesta.
- **No se usa un modelo sin think time:** “un modelo de carga sin think time es una prueba de estrés disfrazada” (S13). La demo de clase lo muestra a propósito (§7).

Volumen esperado por operación (hora punta y pico):

Operación	%	ops/s (hora punta)	Peticiones por operación	req/s hora punta	req/s pico (× 3)	Peticiones en 15 min de carga
Login	1 por sesión	0.83	1	0.83	2.50	750

Operación	%	ops/s (hora punta)	Peticiones por operación	req/s hora punta	req/s pico (× 3)	Peticiones en 15 min de carga
Saldo y perfil	30	1.50	2	3.00	9.00	2,700
Movimientos	15	0.75	1	0.75	2.25	675
Transferir	15	0.75	2 (comisión + POST)	1.50	4.50	1,350 (675 POST)
Contactos	10	0.50	2	1.00	3.00	900
Recarga	10	0.50	2 (configuración + POST)	1.00	3.00	900 (450 POST)
Consulta de deudas	6	0.30	2	0.60	1.80	540
Pago	4	0.20	3 (empresas + deudas + POST)	0.60	1.80	540 (180 POST)
Otras (5 × 2 %)	10	0.50	1	0.50	1.50	450 (90 por endpoint)
Total	100	**5.83**	11.74 por sesión	**9.78**	**29.35**	**8,805**

Con 90 muestras, el p95 de cada endpoint de “otras” es poco estable: el veredicto usa el grupo “otras” (450 muestras) y cada endpoint se reporta como información.

Fases de cada escenario (etiqueta `fase` en cada petición; los umbrales de k6 solo miran la fase medida, en segundos desde el inicio):

Escenario	Fases
Carga	`calentamiento` 0–120 · **`estable` 120–1,020** · `drenaje`
Pico	`calentamiento` 0–120 · **`base` 120–240** · **`salto` 240–540** (12:00–12:05) · `resto` 540–840 · **`recuperacion` 840–1,140** (12:10–12:15) · `drenaje`. Campaña = 240–1,140
Estrés	`base` 0–180 · **`rampa` 180–1,560** · **`meseta` 1,560–1,680** · `drenaje`
Resistencia	`calentamiento` 0–120 · **`estable` 120–3,720** · `drenaje`

Datos de prueba (un usuario distinto por VU, con saldo suficiente). `tests/performance/preparar.sh 3000` generó **3,000 usuarios** (M-16) con los endpoints de soporte: `POST /api/pruebas/reiniciar` (200 usuarios de carga de la semilla, PIN 2580, 1,000 movimientos cada uno), `POST /api/usuarios` para los 2,800 restantes (DNI y celular sintéticos), `PUT /api/pruebas/saldo` (S/ 10,000 a cada uno) y login + 5 contactos por la API. Solo lo que la API no permite se hizo por SQL: clonar el historial de 1,000 movimientos y crear 30 deudas por usuario en un suministro propio. Resultado (`entorno/preparacion.json`): 3,000 usuarios, 15,000 contactos, 90,000 deudas, 3,000,031 movimientos, `integrity_check ok`, base de 539.5 MB, generada el 11-nov-2026 (el mismo día de las corridas, porque los vencimientos y la ventana de 30 días dependen de la fecha).

Operación	Dato por VU	Cómo se garantiza	Regla que se evita romper
Login	Usuario propio y PIN correcto	VU n → fila n de `usuarios.csv` (`exec.vu.idInTest`); no se usan los usuarios 1 a 6 de la semilla (uno está bloqueado)	RN-04 (bloqueo tras 3 fallos)
Transferir	Destino que sea otro usuario Qolqe; monto	`contacto_id` al azar entre sus 5 contactos; monto uniforme de S/ 1.00 a S/ 20.00	RN-05 (≤ S/ 500), RN-06 (S/ 2,000 al día; peor caso por VU ≈ S/ 491), RN-07
Recarga	Número y monto	Número `95XXXXXXX` aleatorio en cada recarga; monto de {5, 10, 20, 30, 50, 100}	R-G1-01, R-G1-03 (5 al día por número)
Deudas y pago	Suministro con deudas que solo use ese VU	Suministro propio con **30 deudas**; se paga la primera pendiente (peor caso ≈ 12.5 pagos por VU)	404 `SIN_DEUDA_PENDIENTE`, 409 `DEUDA_YA_PAGADA`
Contactos	5 contactos y un alias que buscar	Anillo de 5 contactos; se busca el alias de uno de ellos	R-G4-03 (≤ 15 contactos)

Operación	Dato por VU	Cómo se garantiza	Regla que se evita romper
Movimientos	El mismo historial para todos	1,000 movimientos en 120 días (≈ 243 en 30 días), clonados a los usuarios creados por API	Que el N+1 de F-06 no cueste menos justo con más carga
Saldo	Suficiente	S/ 10,000; gasto esperado por VU en el peor caso (62 min) ≈ S/ 2,000	RN-08 (`SALDO_INSUFICIENTE`)

Reglas de uso: cada corrida restaura `base_perf.db` y arranca un contenedor nuevo; nunca se llama a `reiniciar` entre corridas; ninguna corrida cruza la medianoche de Lima. Un VU ejecuta una sola sesión a la vez: **nunca hay dos sesiones simultáneas sobre el mismo usuario** (enunciado §5), lo que también evita provocar DEF-21.

---

5. Criterios de aceptación

"Tiempo de respuesta" es `http_req_duration` de k6 (envío + espera + recepción), la definición de S14: "desde que el cliente envía una petición hasta que recibe la respuesta completa". Los percentiles se calculan por **rango más cercano** (S14). Cada umbral tiene métrica + percentil + valor límite + carga.

Criterio	Umbral aceptable	Fuente del umbral
Tiempo de respuesta (U-01): consultas e inicio de sesión	**p95 < 800 ms** en cada endpoint de consulta, en el login y en el grupo `consulta`, en el estado estable con **100 usuarios simultáneos** (3,000 sesiones/h)	"Consultas e inicio de sesión: el 95 % debe responder en menos de 800 ms". Carga: S-01; qué es consulta: S-02; por endpoint y por grupo: S-03
Tiempo de respuesta (U-02): operaciones de dinero	**p95 < 1,500 ms** en `POST` transferencias, recargas y pagos, y en el grupo `dinero`, con 100 simultáneos	"Operaciones de dinero (transferencias, recargas y pagos): el 95 % debe responder en menos de 1.5 segundos". S-01, S-03
Tiempo de respuesta (U-06): pico de la campaña	De 12:00 a 12:15, pico de **300 simultáneos**: p95 consultas **< 1,600 ms** y dinero **< 3,000 ms**; error ≤ 1 %; 0 sesiones descartadas; en 12:10–12:15 se vuelve a cumplir U-01 y U-02	"Durante el pico de la campaña se acepta una degradación razonable del servicio". S-05, S-07, S-08
Tasa de error máxima (U-03)	**≤ 1 %** de operaciones de negocio fallidas (login incluido) con 100 simultáneos. Error de rendimiento = 5xx, timeout de 30 s, error de conexión, 2xx con cuerpo incorrecto o 401 a mitad de sesión. Los 4xx de regla de negocio van aparte (V-02)	"En hora punta, no más del 1 % de las operaciones puede fallar". S-06; enunciado §5 ("Sepárenlos en el análisis")
Throughput mínimo (U-04)	En el estado estable: **≥ 9.6 req/s atendidas** (98 % de las 9.78 ofrecidas, M-08) y **≥ 49 sesiones completadas por minuto** (98 % de 50)	"…la plataforma debe sostener el volumen de la hora punta…". S-04
Sin acumular retrasos (U-05)	(a) **0 sesiones descartadas** (`dropped_iterations`); (b) **ninguna ventana de 1 min** con p95 sobre U-01 o U-02; (c) simultáneos medidos entre **90 y 110** por ventana	"…sin acumular retrasos". S-04

Criterio	Umbral aceptable	Fuente del umbral
Integridad del dinero (U-07)	**0 descuadres** en los 6 chequeos de saldos y movimientos antes y después (C1–C6, §6), en todas las corridas	"…ninguna operación de dinero puede quedar a medias ni aplicarse dos veces". S-09; enunciado §5
Capacidad (U-08)	Se **mide**: máximo de simultáneos que cumple U-01 a U-05 en ventanas de 30 s antes de la primera falla sostenida (2 ventanas seguidas), redondeado hacia abajo a 50	"…hasta cuántos usuarios simultáneos podemos crecer antes de dejar de cumplir". S-10
Operación continua (U-09)	En 60 min a 100 simultáneos: (a) las 12 ventanas de 5 min cumplen U-01 a U-03; (b) RSS proyectado a 54,000 sesiones **< 700 MB** (70 % de 1 GB, S13); (c) hilos y descriptores proyectados **≤ 2 ×** el inicial; (d) p95 de los últimos 10 min **≤ 1.2 ×** el de los primeros 10	"Qolqe opera todos los días de 6:00 a 24:00 sin reinicios". S-11, S-12

- **Por qué U-07 es 0 y no < 1 %.** S14 advierte que un 0 % de error "casi nunca es alcanzable", pero eso es sobre disponibilidad. Una operación de dinero a medias o duplicada es un defecto de integridad y el requerimiento la prohíbe sin tolerancia.
- **CPU < 80 % y memoria < 70 % (S13) no son SLA**: el requerimiento no los pide. Se usan como alertas de diagnóstico; el de memoria, además, en U-09(b).

Partes del requerimiento que no se podían convertir en umbral y supuesto tomado:

ID	Qué falta o es ambiguo	Supuesto tomado	Justificación
S-01	Los umbrales de tiempo **no dicen con qué carga** se cumplen	Se exigen en hora punta: 100 simultáneos, 3,000 sesiones/h	Única carga "normal" del requerimiento; en la campaña rige U-06
S-02	"Consultas" no dice qué peticiones incluye	Toda petición GET de la mezcla, incluidas las GET de apoyo de una operación de dinero	Responden datos sin mover dinero
S-03	"El 95 %" puede ser del grupo o de cada operación	Veredicto **por endpoint** y también por grupo	Un p95 de grupo puede pasar mientras un endpoint (movimientos, ≈ 9 % del grupo) falla para todos sus usuarios
S-04	"Sin acumular retrasos" no es un número	Modelo abierto; retraso = la cola crece. Se mide con U-04 y U-05	En un modelo cerrado el retraso queda oculto. El 98 % deja ≈ 5 σ de margen a la variación de la mezcla
S-05	"Degradación razonable" no es un número	p95 hasta **2 ×** el umbral normal (1,600 / 3,000 ms); error ≤ 1 %; 0 rechazadas; recuperación	Se acepta degradación de velocidad, no fallos. Con 2 × sigue bajo 3 s (aporte propio; Nielsen, 1993). Se reporta también contra el umbral normal (paso 4 de S14)
S-06	"Operaciones que fallan" no está definido	Unidad = **operación de negocio**; error de rendimiento: 5xx (incluido `database is locked`), timeout de 30 s, conexión, 2xx con cuerpo incorrecto, 401 con token vigente	Enunciado §5 y S14 (5xx, timeouts, "respuestas incorrectas")
S-07	"El triple de usuarios simultáneos": ¿máximo o promedio?	Es el **máximo**: 3 × 100 = **300**	Cuadra con "la mayoría inicia sesión en los primeros 5 minutos"; los 300 sostenidos los cubre el estrés
S-08	"La mayoría" no es un número	150 sesiones/min de 12:00 a 12:05 y 50/min hasta 12:15: el **58 %** empieza en los primeros 5 min	Es la lectura que menos inventa

ID	Qué falta o es ambiguo	Supuesto tomado	Justificación
S-09	“A medias o dos veces” no dice cómo se verifica	Saldos y movimientos antes y después (C1–C6) cruzados con los 201 de k6; k6 **no reintenta** POST	Enunciado §5. Sin reintentos, un duplicado solo puede venir del servidor
S-10	“Dejar de cumplir” no dice con qué criterio	Los de hora punta (U-01 a U-05) en ventanas de 30 s	Responde “hasta cuántos podemos crecer” con el SLA del día normal
S-11	Solo se conoce el volumen de la hora punta	Cota de un día: 18 h × 3,000 = **54,000 sesiones**	Conservador: ninguna hora tiene más carga que la hora punta
S-12	No se pueden correr 18 h antes de la entrega	Resistencia de **60 min** en hora punta y proyección	Se declara en §6
S-13	La sesión dura 2 min, pero 6 pausas × 18 s = 108 s	**12 s finales sin peticiones**	No agrega carga; hace que “simultáneos” signifique lo mismo que en el requerimiento (L = 100)
S-14	Uniforme 10–25 tiene media 17.5 s, no 18	**Triangular(10, 19, 25)**: rango 10–25 y media 18 s	Cumple los dos datos con un solo supuesto (la moda)
S-15	Para pagar hay que consultar antes	El 4 % de pagos **incluye** su consulta; el 6 % solo consulta	Así funciona la pantalla `/servicios`
S-16	Peticiones por operación y cierre de sesión	Peticiones de cada pantalla según `app.js` (M-07); **sin logout**	La sesión es login + 6 operaciones; el token vence a los 30 min
S-17	La capacidad de producción es desconocida	Los resultados son una **línea base relativa**	Enunciado §5 y S13 (“no extrapolar”). Ver §6

Criterios de validez de cada corrida (no son umbrales del SLA; si uno no se cumple, la corrida no se analiza, se mueve a `resultados/descartadas/` con su motivo y se repite):

ID	Criterio	Fuente
V-01	**CPU de k6 < 80 %** del equipo en todas las ventanas (decide). El CPU total del equipo se informa en una fila aparte que no invalida (ver cambios abajo)	Enunciado §5: “si supera ~80 %, el resultado no es válido”
V-02	Errores de datos (4xx de negocio) **= 0**; si superan el 0.1 % se corrigen los datos y se repite	Enunciado §5
V-03	Mezcla lograda a **± 2 puntos porcentuales** de la pedida	Rúbrica: “operaciones ponderadas”
V-04	Equipo libre antes de empezar: **CPU ocioso ≥ 65 %** (`top`, 3 s); la carga de 1 min se registra	Enunciado §5; S13 (“un servidor compartido introduce ruido”)
V-05	Base restaurada idéntica a `base_perf.db` (huella), proceso nuevo y `cpu.max` esperado	Corridas comparables
V-06	Con 1 VU, el p95 de cada endpoint **≤ 10 %** de su umbral	Enunciado §5 (referencia de 1 VU)
V-07	Sin avisos de k6 “Insufficient VUs” en carga y resistencia	Que el generador sostenga la tasa

Cambios de criterio hechos durante la ejecución (declarados).

- **V-04** (11-nov, commit `5adb217`): el diseño pedía carga promedio de 1 min < 1.5, pero en esta Mac no baja de ~4 aunque no haya nada corriendo (al empezar las corridas se registró entre 4.23 y 12.01, `corrida.json`). Se cambió a CPU ocioso ≥ 65 % (medido: 69.4 % a 93.3 %). Los dos valores quedan en `corrida.json`.
- **V-01** (cambio hecho **después de ver los datos**): el diseño pedía CPU de k6 **y** CPU total del equipo < 80 %. Se dejó decidir solo al CPU de k6, que es el criterio del enunciado (“vigilen también el CPU de la herramienta”); el CPU total del equipo pasó a fila informativa con los minutos en que superó el 80 %. Pasó en dos corridas por procesos de macOS (`mediaanalysisd`, `suggestd`), nunca por k6: en el pico (90.2 % en las ventanas de los minutos 10.25–11.25; el p95 del login subió de ~85 a 145 ms, lejos de 1,600) y en el estrés (82.3 % en el minuto 2.2, todavía en la base de 100 usuarios, 10 min antes de la rodilla). Con el criterio original esas dos corridas se habrían repetido.
- **V-06** no se cumplió en el login: 92 ms frente a 80 ms en la referencia (91.3 ms según k6), y también falló en la referencia descartada (94 ms). Se aceptó como limitación

del entorno (§6) porque es el 11.5 % del umbral de 800 ms y no cambia ningún veredicto.

---

6. Restricciones del entorno

- Entorno elegido (local / dos equipos / Docker / Codespaces) y especificaciones de la máquina:

Docker: Qolqe en un contenedor con `--cpus=1 --memory=1g --memory-swap=1g` y k6 en macOS, en la misma Mac.

Opción	Evaluación para este caso (una persona, una Mac)	Decisión
Local	Qolqe tendría los 10 núcleos: llegar a su límite exigiría miles de VUs en el mismo CPU que k6, y la herramienta competiría con la aplicación (regla del 80 %). macOS trae `somaxconn = 128` y `maxfiles = 256`	No
Dos equipos	No hay una segunda laptop (trabajo individual)	No aplica
Codespaces	La latencia de internet se suma al tiempo de respuesta; entorno compartido; la demo dependería del Wi-Fi del aula	No
Docker	La cuota fija 1 CPU para Qolqe y deja 9 núcleos a k6: no compiten. Límite reproducible; `docker stats` y `cpu.stat` miden el contenedor; el límite se alcanza con VUs manejables, así que el estrés lo encuentra. Es la opción de ejemplo del enunciado y la misma imagen sirve para la demo	**Sí**

Especificaciones (`resultados/entorno/entorno.json`, `cgroup.txt`, `pip_freeze.txt`):

- Equipo: MacBook Pro (MacBookPro18,3), Apple M1 Pro de 10 núcleos (8 de rendimiento y 2 de eficiencia), 16 GB de RAM, macOS 27.2.1.
- Docker Desktop: motor 29.8.2; VM de 10 CPU y 8.3 GB; kernel 7.0.14-linuxkit; cgroup v2.
- Qolqe: imagen `qolqe-perf:p3` (`python:3.11.15-slim-bookworm`, arm64), un solo proceso de uvicorn 0.54.0 (como pide el README: el reloj de pruebas vive en memoria), FastAPI 0.117.1; SQLite en modo WAL en el volumen `qolqe-perf-datos`; publicado en `127.0.0.1:8090`.
- Límites comprobados en cada corrida (V-05, `corrida.json`): `cpu.max = 100000 100000` (1 CPU; 200000 en la escalabilidad), `memory.max = 1073741824` (1 GB).
- Generador: k6 v2.3.0 (go1.27.1, darwin/arm64) en el host. Medición y análisis: Python 3.11.6, psutil 7.2.2, py-spy 0.4.3 (en la imagen), pandas, numpy y matplotlib.

Condiciones de la máquina en cada corrida (`corrida.json` y `veredicto.md`, V-01 y V-04):

Corrida	Carpeta	CPU ocioso al empezar	Carga de 1 min	k6, peor ventana (% del equipo)	Equipo, peor ventana
Referencia	`referencia/20261111-1828`	82.6 %	5.40	0.1 %	21.0 %
Calibración	`calibracion/20261111-1830`	81.4 %	6.55	0.1 %	26.4 %
Carga	`carga/20261111-1833`	81.5 %	4.23	0.3 %	60.9 %
Pico	`pico/20261111-1918`	81.4 %	5.05	0.3 %	**90.2 %** (min 10.25–11.25)
Estrés 1 CPU	`estres/20261111-1940`	80.3 %	5.05	1.2 %	**82.3 %** (min 2.2, base)
Resistencia	`resistencia/20261111-1958`	69.4 %	12.01	0.2 %	26.6 %
Estrés 2 CPU	`escalabilidad/20261111-2103`	93.3 %	5.73	1.7 %	42.6 %
Demo	`demo/20261111-2125`	90.4 %	6.80	0.5 %	28.1 %

- Cómo se midieron CPU y memoria:

La herramienta no mide el servidor, así que se midió aparte, **sobre el proceso de Qolqe**, con tres monitores que escriben una fila por segundo durante **cada** corrida (`tests/performance/correr.sh` los arranca y los detiene):

Archivo	Qué mide	Cómo
`recursos_qolqe.csv`	CPU (% de 1 núcleo) y RSS (MB) del proceso de Qolqe, hilos, descriptores, CPU del cgroup y throttling acumulado (`nr_throttled`, `throttled_usec`)	Script psutil del enunciado (`monitor/monitor.py`, mismas columnas `hora,cpu_%,memoria_MB`) sobre el PID 1 del contenedor, desde un contenedor auxiliar con `--pid=container:qolqe-perf --cgroupns=host` para no gastar la cuota de Qolqe
`recursos_docker.csv`	CPU del contenedor (100 % = la cuota), memoria y procesos	`docker stats` (`monitor/monitor_docker.py`)
`recursos_k6.csv`	CPU y memoria de **k6** y CPU total del equipo (V-01)	psutil sobre el PID de k6 en el host; el CPU se divide entre 10 núcleos

- **Hora de inicio y fin** de cada prueba en `corrida.json` (hora de Lima, ISO y epoch). Todos los CSV llevan `ts` (epoch) para cruzar recursos y muestras de k6 en el mismo eje.
- `cpu.stat` del cgroup antes y después de cada corrida (CPU consumido y throttling). En la calibración, ΔCPU ÷ 40 sesiones da D_sesión (M-14).
- En el estrés, perfil de CPU con **py-spy** (60 s en la base y 60 s cerca de la rodilla) y foto de los hilos (`perfil_base.svg`, `perfil_saturacion.svg`, `hilos_meseta.txt`).
- Ventanas del análisis: 1 min en carga, 30 s en pico y estrés, 5 min en resistencia.

Verificación del dinero antes y después (U-07). Cada corrida restaura `base_perf.db`, así que "antes" = `base_perf.db` (V-05 lo comprueba) y "después" = `qolqe.db` sin tráfico. `integridad/verificar.py` corre en solo lectura y escribe `verificacion_dinero.txt` con la línea `TOTAL descuadres: N`. Se probó antes con casos negativos: un duplicado y un saldo negativo provocados salieron como FALLA.

Chequeo	Qué compara	Correcto
C1 Conservación	Σ saldos antes − Σ saldos después = recargas + pagos + comisiones nuevas	descuadre 0
C2 Libro mayor	Por usuario: Δ saldo = Σ importes de sus movimientos nuevos	0 filas
C3 Operación ↔ movimientos	Cada transferencia tiene 1 ENVIADA + 1 RECIBIDA; recarga y pago, 1 movimiento; sin huérfanos	0 filas
C4 Pagos	Ninguna deuda pagada dos veces; toda deuda PAGADA tiene su pago y al revés	0 filas
C5 Reglas	Ningún saldo negativo ni suma diaria sobre el límite (síntomas de DEF-21)	0 filas
C6 Conteos	Filas nuevas por tipo frente a los 201 de k6: el exceso solo puede ser respuestas perdidas (timeout) o POST cortados al final, y se explica	diferencia explicada

- Limitaciones conocidas:
  - **Misma laptop.** k6 y Qolqe comparten el equipo; la cuota de 1 CPU los separa, pero el proxy de puertos de Docker Desktop suma latencia a Qolqe y CPU al host. Por eso la referencia de 1 VU se mide en Docker (V-06) y se vigila el CPU de k6 (V-01).
  - **Procesos de macOS en segundo plano.** `mediaanalysisd`, `suggestd` y otros movieron el CPU total del equipo por encima del 80 % en algunas ventanas del pico y del estrés (tabla de arriba). No fue k6 (≤ 1.7 % por ventana en todas las corridas). En el pico coincidió con una subida del p95 del login a 145 ms.
  - **Referencia de 1 VU (V-06).** El login dio 91–92 ms frente al límite de 80 ms en las dos referencias. Explicación probable: con 1 VU el PBKDF2 corre en una vCPU casi ociosa que macOS programa en núcleos de eficiencia. Por lo mismo, la calibración sobrestima el CPU por sesión: 157 ms con 1 VU frente a ~101–106 ms bajo carga (§8).

    - `--cpus=1` es una **cuota**, no un núcleo dedicado: dentro del contenedor `nproc` da 10 y los hilos que calculan PBKDF2 en paralelo agotan la cuota antes de tiempo (throttling, registrado en `nr_throttled`).
    - La memoria del cgroup incluye la caché de páginas de la base (≈ 540 MB): la memoria del proceso se toma como RSS con psutil.
    - SQLite con un solo proceso de uvicorn, igual que el entorno entregado: no representa una base de datos de servidor ni varios procesos.
    - Datos sintéticos (3,000 usuarios con historial clonado); no hay datos de producción.
    - La resistencia dura 60 min y no 18 h (S-12): las 18 h se proyectan.
    - El estrés no llegó a la meseta: k6 lo cortó por la red de seguridad del 20 % (ver abajo).
- Diferencias entre el plan y la ejecución real:
    - **Estrés cortado por la red de seguridad.** k6 corta si más del 20 % de las operaciones falla: con 1 CPU cortó a los 952 s (1,103 simultáneos, 20.05 % de operaciones fallidas, código 99) y con 2 CPU a los ~1,315 s (1,656). La rodilla se pasó con claridad, pero no hubo meseta ni foto de hilos con 1 CPU.
    - **Cima del estrés:** la calibración sugirió L_top = 1,100; se usó 2,400 = máx(1,100; 2,400), porque en el ensayo previo el CPU por sesión bajo carga fue ≈ 100 ms y no 157 ms (regla de `tests/performance/README.md` §4).
    - **py-spy adelantado:** el perfil de saturación se tomó a 790–850 s con 1 CPU y a 1,000–1,060 s con 2 CPU, y la foto de hilos a 1,250 s con 2 CPU, para que ocurrieran antes del corte. Con 1 CPU el perfil cayó ya dentro del colapso (empezó en el minuto ~12.2).
    - **Cambios de criterio V-04 y V-01** (detalle en §5) y harness: `corrida.json` registra los commits `5adb217` (referencia, calibración, carga) y `cba543f` (resto) con cambios del harness aún sin commit; quedaron guardados en `cba543f` y `3b57db2`. Ninguno cambia la carga ni `qolqe/`.
    - **Corridas descartadas** (`resultados/descartadas/`, cada una con `motivo.txt`): `referencia_20261111-1826` (V-06 falló en login, 94 ms, y movimientos, 91 ms, con procesos de macOS al ~150 % de CPU; se repitió con el equipo más quieto) y `pico_20261111-1852` (Docker Desktop recibió una orden de salir a las 19:12:52, en el drenaje y después de todas las fases medidas; el dinero se verificó a mano con 0 descuadres; se repitió completo).
    - **Opcionales:** se corrió la escalabilidad (estrés con `--cpus=2`, `resultados/escalabilidad/`); no se corrieron la confirmación de capacidad sostenida (`L_CAP`) ni el contraste con JMeter.
    - Puerto `127.0.0.1:8090` (el diseño decía 8141); resistencia de 60 min en lugar de 18 h. Los ensayos comprimidos (`ESCALA` < 1) de antes de las corridas **no son evidencia** y no se usan.
- Línea base relativa:

  Los resultados son una **línea base relativa** de Qolqe en un contenedor de 1 CPU y 1 GB dentro de una laptop. **No son la capacidad de producción.** Sirven para comparar versiones y configuraciones (por ejemplo, 1 contra 2 CPU) y para ubicar el primer cuello de botella. No deben extrapolarse (“soporta N usuarios en la laptop, entonces soporta N en producción”, S13).

---

7. Resultados

Cada prueba deja su evidencia en `tests/performance/resultados/<escenario>/<AAAAMMDD-HHMM>/`: `corrida.json`, `reporte.html` (reporte de k6), `resumen.json`, `muestras.csv.gz`, `recursos_*.csv`, `verificacion_dinero.txt`, `tabla_resumen.md`, `veredicto.md` y `graficos/`. El veredicto binario por umbral se da **antes** de mirar los gráficos (paso 1 de S14) y los errores de rendimiento y de datos se reportan por separado. Todos los p95 están en ms, por rango más cercano.

Corridas de apoyo: referencia de 1 VU y calibración

- **Referencia** (`referencia/20261111-1828`, 18:29:02–18:30:09): p95 con 1 VU del login **92 ms** (límite V-06: 80 ms, **no cumple**), `POST /api/transferencias` 136 ms (límite 150 ms, cumple), movimientos 35 ms y el resto de consultas ≤ 19 ms. 0 errores de rendimiento y 0 de datos; U-07: 0 descuadres.
- **Calibración** (`calibracion/20261111-1830`, 40 sesiones de 1 en 1): D_sesión = **157 ms** de CPU por sesión → L_sat = 120 ÷ 0.157 = **763** → L_top sugerido = **1,100**. 0 errores; U-07: 0 descuadres.

Prueba 1: Carga — hora punta, 100 usuarios simultáneos (50 sesiones/min), 15 min de estado estable

Evidencia: `resultados/carga/20261111-1833/` · inicio 18:33:22 · fin 18:52:32 · código de salida de k6: **0** (todos los umbrales cumplen). **Corrida válida** (V-01 k6 0.3 % por ventana; V-02 0; V-03 peor operación a 0.78 pp; V-04 81.5 % ocioso; V-05 base idéntica; V-07 0 avisos).

Umbral	Límite	Medido (fase `estable`, 900 s)	¿Cumple?
U-01 consultas e inicio de sesión (grupo)	p95 < 800	93	✓ Cumple
U-01 login / endpoint de consulta más lento sin login	p95 < 800	105 / movimientos 38	✓ (incluido arriba)
U-02 dinero (grupo)	p95 < 1,500	128	✓ Cumple
U-02 transferir / recargar / pagar	p95 < 1,500	134 / 13 / 12	✓ (incluido arriba)
U-03 operaciones fallidas	≤ 1 %	0.00 % (5,245 operaciones)	✓ Cumple
U-04 throughput	≥ 9.6 req/s y ≥ 49 sesiones/min	9.77 req/s (ofrecidas 9.78); 49.6 sesiones/min	✓ Cumple
U-05 sin acumular retrasos	0 descartadas; 0 ventanas sobre umbral; 90–110 simultáneos	0; 0 de 15 (peor: consulta 101, dinero 141); 98.8–101.0	✓ Cumple
U-07 integridad del dinero	0 descuadres	0 (BD = 201 recibidos: 730 transferencias, 519 recargas, 212 pagos)	✓ Cumple

Tabla resumen por endpoint, fase estable (formato S14; ms; `carga/20261111-1833/tabla_resumen.md`):

Endpoint	Peticiones	Promedio	p50	p90	p95	p99
`login` POST /api/auth/login	750	91	93	103	105	109
`saldo` GET /api/cuentas/me/saldo	1,339	7	6	11	13	16
`perfil` GET /api/cuentas/me	1,339	3	3	5	6	8
`movimientos` GET /api/movimientos	689	25	22	36	38	46
`comision` GET …/transferencias/comision	639	6	6	11	12	16

Endpoint	Peticiones	Promedio	p50	p90	p95	p99
`contactos_listar` / `contactos_buscar`	483 / 483	8 / 4	7 / 4	13 / 6	14 / 8	19 / 10
`recargas_config` / `empresas` / `deudas`	463 / 449 / 449	4 / 6 / 4	3 / 5 / 4	8 / 11 / 7	9 / 12 / 7	14 / 17 / 11
otras: `limites`, `alertas`, `solicitudes`, `dispositivos`, `sesiones`	83–94 c/u	6–7	6	10–12	12	14–21
`transferir` POST /api/transferencias	639	96	95	128	134	144
`recargar` POST /api/recargas	463	9	8	12	13	18
`pagar` POST /api/servicios/pagos	178	8	7	10	12	17
grupo consulta	7,516	16	5	72	**93**	103
grupo dinero	1,280	52	25	120	**128**	140
total (0.00 % de error; 9.77 req/s)	8,796	21	6	85	101	126

Errores de rendimiento: ninguno. Errores de datos (4xx, V-02): 0. Mezcla lograda (V-03): todas las operaciones a ≤ 0.78 pp de la pedida (transferir 14.2 % frente a 15 %).

Resultado redactado (modelo de S13): “Con 100 usuarios simultáneos (3,000 sesiones/h, perfil operacional de la hora punta), el p95 de `POST /api/transferencias` fue 134 ms frente al umbral de 1,500 ms (cumple) y el del login fue 105 ms frente a 800 ms (cumple); la tasa de error fue 0.00 %, frente al umbral de 1 %.”

- **Recursos** (CPU y memoria de Qolqe y de la herramienta):

Proceso	CPU promedio	CPU máximo	Memoria	Otros
Qolqe (% de 1 CPU, psutil)	12.0 % en el estable (10.9 % toda la corrida)	13.3 % por ventana; 23.0 % en 1 s	RSS 56.0 → 62.7 MB	hilos máx. 11; descriptores máx. 56; throttling 2 períodos (0.009 s)
k6 (% del equipo, 10 núcleos)	0.19 %	0.28 % por ventana; 1.4 % en 1 s (14 % de un núcleo)	152 MB máx.	V-01 < 80 %: ✓
Equipo completo	35.5 %	60.9 % por ventana (100 % en un pico de 1 s)	—	informativo

- **Gráficos** (tiempo de respuesta, errores, throughput y recursos a lo largo de la prueba):

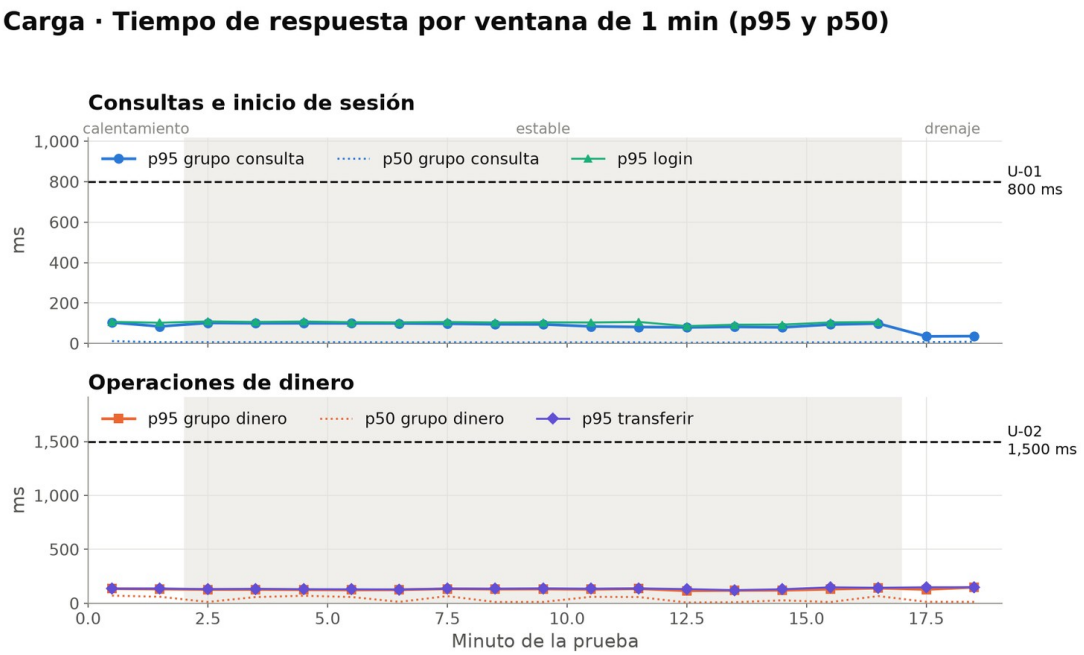


Figura 1. Carga: p95 y p50 por ventana de 1 min de consultas, login, dinero y transferir, con los umbrales U-01 y U-02 (`carga/20261111-1833/graficos/G1_tiempo_respuesta.png`).

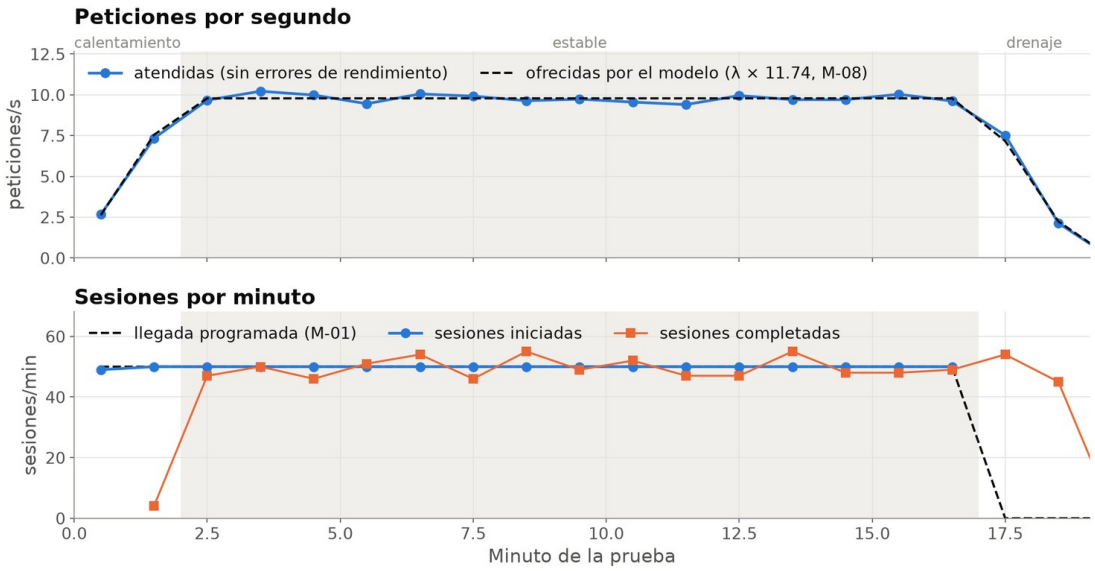


Figura 2. Carga: throughput atendido frente al ofrecido por el modelo (M-08) (`G2_throughput.png`).

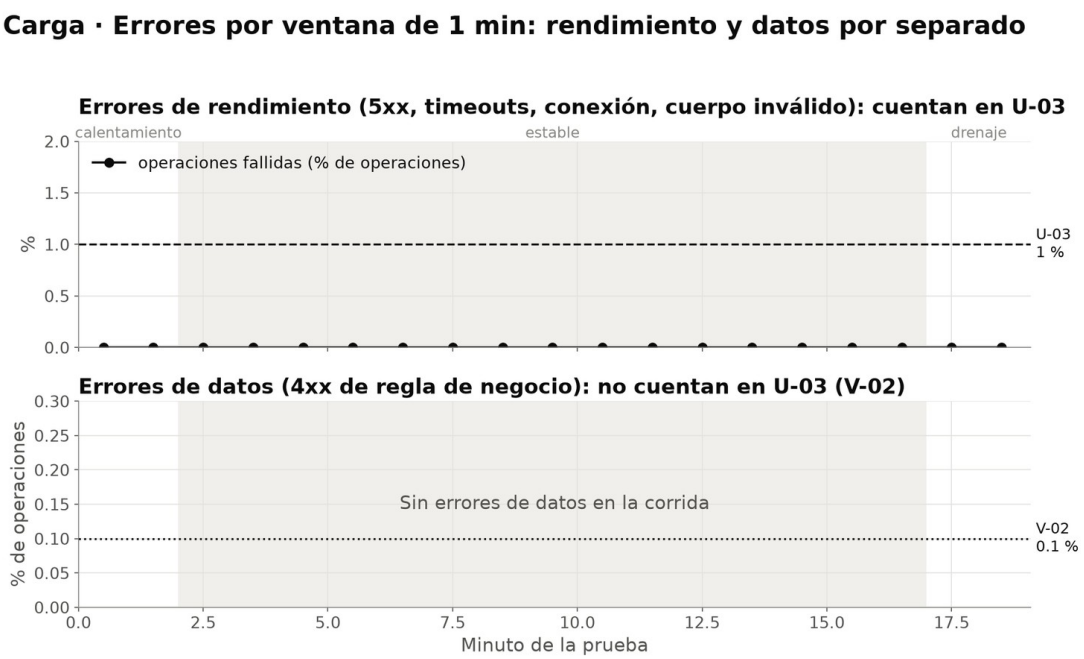


Figura 3. Carga: errores de rendimiento y de datos por separado, con la línea del 1 % (`G3_errores.png`).

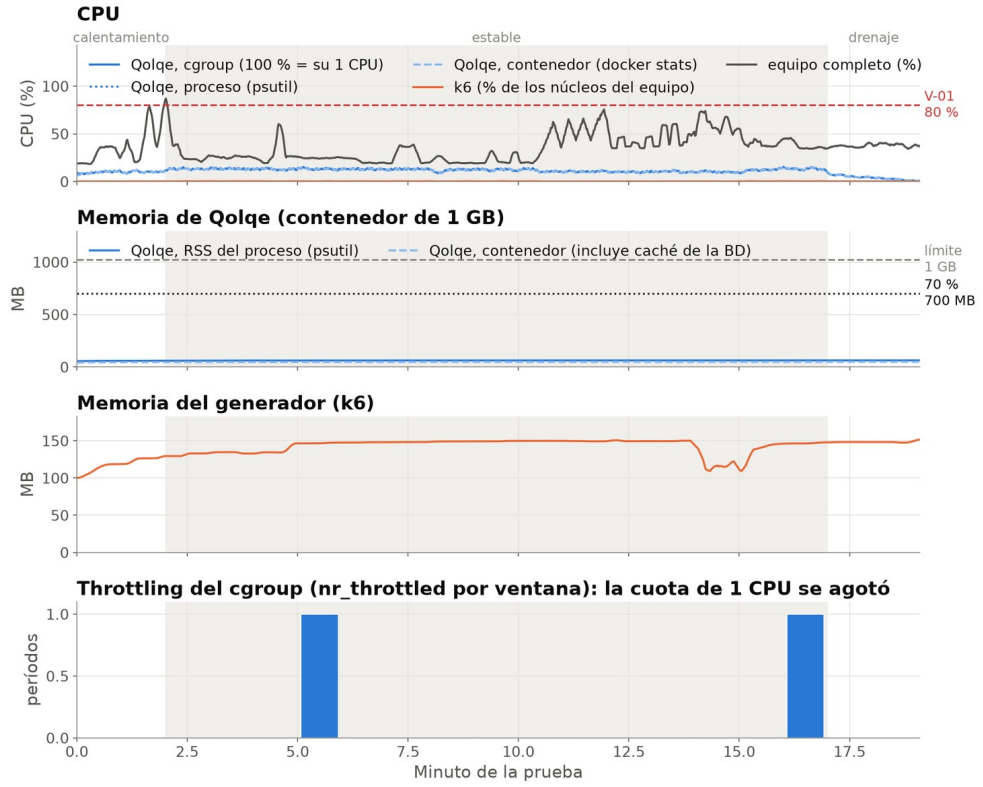


Figura 4. Carga: CPU y RSS de Qolqe, CPU y memoria de k6, CPU del equipo con la línea del 80 % y throttling (`G5_recursos.png`).

Lectura: el p95 de consultas se mantiene entre ~80 y ~100 ms y el de dinero entre ~120 y ~140 ms en las 15 ventanas estables, lejos de 800/1,500 ms. Lo atendido sigue a lo ofrecido (9.77 frente a 9.78 req/s) y no hay errores. Qolqe usa el 12 % de su CPU y el RSS queda plano en ~63 MB. Los picos del CPU del equipo son de procesos de macOS y no se reflejan

en el p95. `G4_usuarios_simultaneos.png` muestra 98.8–101 simultáneos, como predice Little.

Prueba 2: Estrés — rampa de 100 a 2,400 usuarios simultáneos (cortada por k6 a los 952 s, con 1,103)

Evidencia: `resultados/estres/20261111-1940/` · inicio 19:40:31 · fin 19:56:23 · código de salida de k6: **99**: la red de seguridad del 20 % cortó la corrida a los 952 s con 1,103 simultáneos (20.05 % de operaciones fallidas). **Corrida válida** (V-01 k6 1.2 % por ventana; V-02 0; V-03 0.94 pp; V-04 80.3 %; V-05 ok). El CPU del equipo pasó del 80 % solo en el minuto 2.2, en la base.

Capacidad (U-08): 850 usuarios simultáneos que cumplen U-01 a U-05 (≈ 850 ÷ 120 s × 3,600 = **25,500 sesiones/h**). Última ventana que cumple: minuto 11.8 con 876 simultáneos; primera falla sostenida: minuto 12.2 con **946**, donde fallan a la vez todos los criterios (p95 de consulta, dinero, login, saldo, movimientos, transferir y recargar; error > 1 %; atendido < 90 % de lo ofrecido). Throughput máximo atendido: **85.2 req/s con 876 simultáneos**. Hasta la capacidad (25,589 peticiones): p95 consulta 75 ms, dinero 121 ms, 0.00 % de error (`comparativo/resumen_escenarios.csv`).

Un momento cada 100 usuarios (formato del ejercicio 3 de S14; `tabla_estres.md`; tiempos en ms):

Mom.	Min	Simult.	p50	p95	p95 cons.	p95 dinero	Error %	req/s	CPU %	RSS MB	Hilos	Throt.
T1	3.2	102	7.5	104	85.2	138	0.0	10.1	13.6	62.0	7.4	1
T2	5.2	226	2.9	82.8	76.1	115	0.0	23.2	26.8	62.8	9.2	4
T3	6.2	329	2.4	87.5	74.3	126	0.0	31.0	33.3	63.8	9.2	0
T4	7.2	427	3.4	90.0	75.8	121	0.0	41.6	46.0	65.1	9.3	11
T5	8.2	528	2.2	80.9	75.0	123	0.0	51.9	48.7	66.3	10.4	4
T6	9.2	623	2.2	81.3	73.9	117	0.0	60.8	56.2	66.3	10.0	7
T7	10.2	730	2.2	82.7	74.3	120	0.0	69.7	65.3	67.5	11.1	18
T8	11.2	822	2.2	84.7	73.1	120	0.0	81.3	72.6	69.7	12.1	25
T9	12.2	**946**	61.9	**13,374**	13,296	18,672	**13.7**	**26.1**	60.0	126	43.1	120
T10	12.8	**1,103**	30,000	**30,002**	30,002	30,004	**92.5**	**5.8**	41.7	244	48.6	80

Errores de rendimiento (toda la corrida): **116 respuestas 500** (`database is locked`) y **4,762 timeouts** de 30 s; errores de datos: 0. U-07: **0 descuadres**; 2 transferencias se aplicaron sin que el cliente recibiera la respuesta (timeout) y quedaron sin respuesta 29 transferencias, 68 recargas y 1 pago: todo concilia en C6 (`verificacion_dinero.txt`).

- **Recursos** (CPU y memoria de Qolqe y de la herramienta):

Proceso	Base (T1: 102)	Última ventana que cumple (876)	Colapso (T9: 946; T10: 1,103)	Otros
Qolqe CPU (% de 1 CPU)	13.6 %	75.5 %	60.0 % → 41.7 %	throttling total 525 períodos (63.5 s)
Qolqe RSS / hilos	62.0 MB / 7.4	71.7 MB / 12	126 → 244 MB / 43–49	máx. de la corrida: 478 MB / 55 hilos / 3,152 descriptores
k6 CPU / memoria	—	—	—	máx. 1.2 % del equipo por ventana; 156 % de un núcleo en 1 s; 1.6 GB
Equipo completo	82.3 % (min 2.2)	—	—	V-01 informativo; no coincide con la rodilla

- **Gráficos** (tiempo de respuesta, errores, throughput y recursos a lo largo de la prueba):

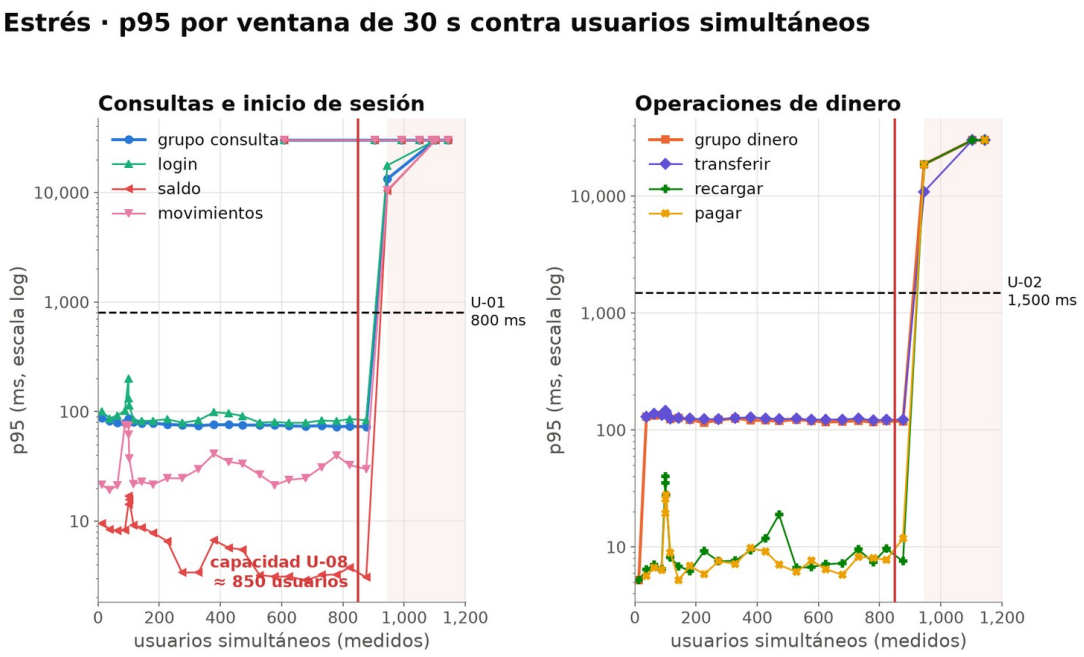


Figura 5. Estrés 1 CPU: p95 por ventana de 30 s contra usuarios simultáneos, con umbrales y la capacidad U-08 (escala log) (`estres/20261111-1940/graficos/G6_p95_vs_usuarios.png`).

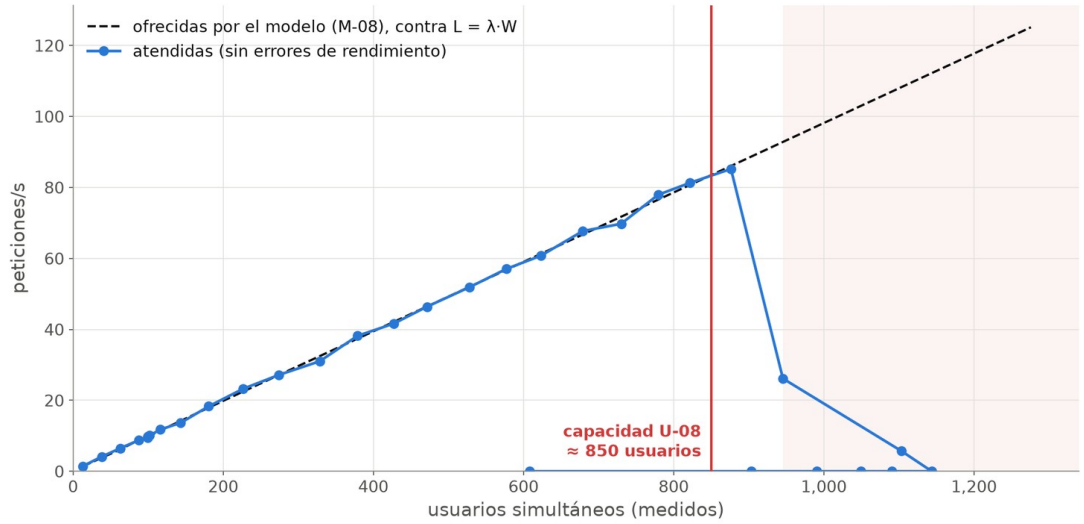


Figura 6. Estrés 1 CPU: throughput atendido contra ofrecido; rodilla de saturación en ~876 usuarios (`G7_throughput_vs_usuarios.png`). Los puntos en 0 req/s son ventanas del drenaje tras el corte de k6 (artefacto del gráfico).

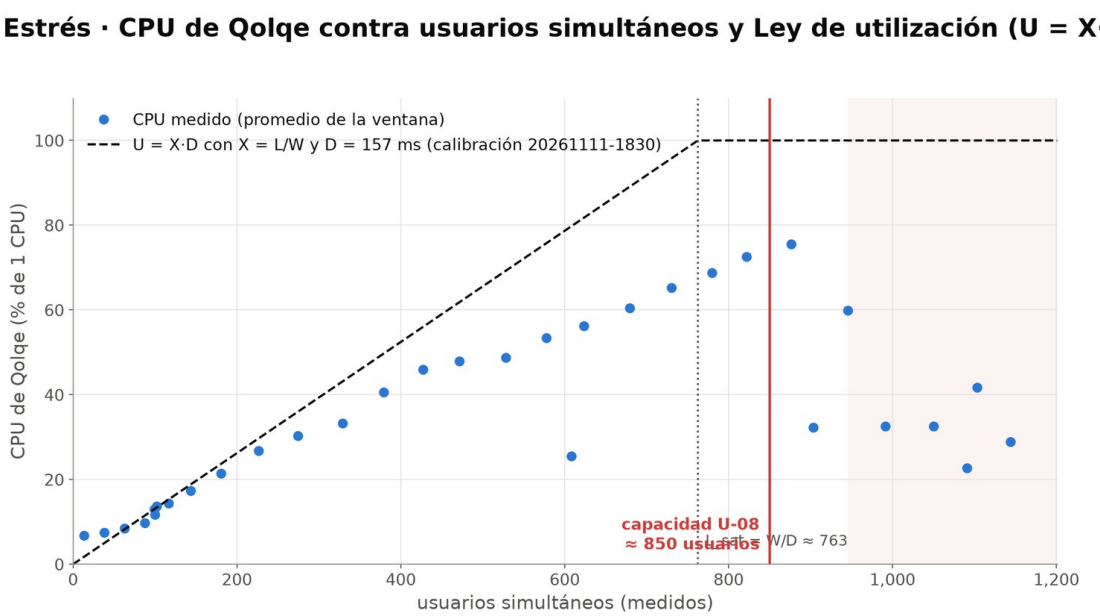


Figura 7. Estrés 1 CPU: CPU de Qolqe contra usuarios y la ley de utilización U = X·D con D = 157 ms (`G8_cpu_vs_usuarios.png`).

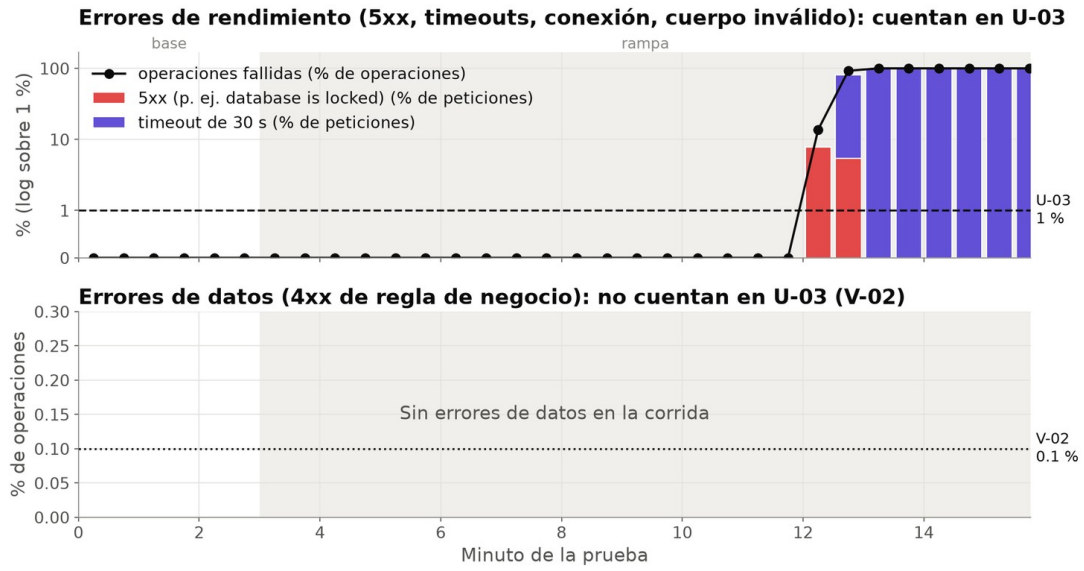


Figura 8. Estrés 1 CPU: errores por ventana de 30 s; primero 500 `database is locked` y luego timeouts de 30 s (`G3_errores.png`).

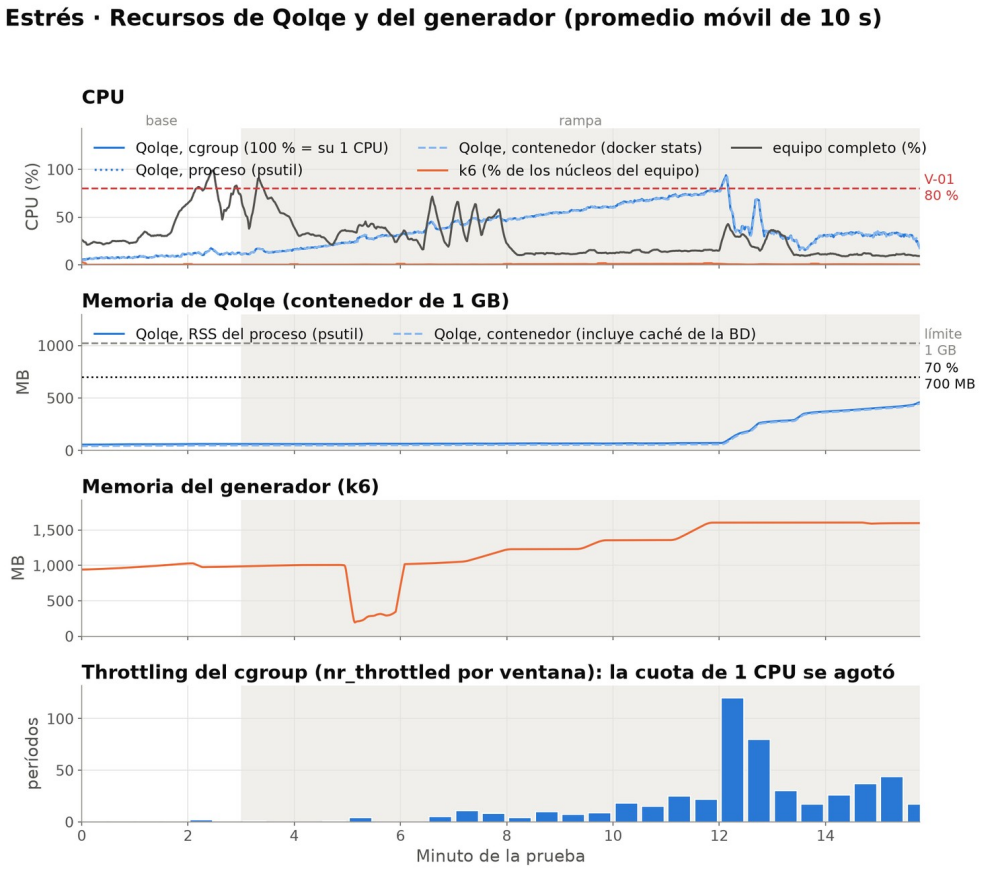


Figura 9. Estrés 1 CPU: recursos de Qolqe y de k6 a lo largo de la prueba (`G5_recursos.png`).

Lectura: el p95 de todos los endpoints críticos se mantiene plano (consulta ~73–85 ms, dinero ~115–138 ms) hasta 876 simultáneos y en la ventana siguiente (946) salta a 13.4 s con 13.7 % de error: **no hay degradación gradual**. Lo atendido sigue la recta de lo ofrecido hasta 85.2 req/s y cae a 26.1 y 5.8 req/s ("los VUs suben pero el throughput baja", S14). El CPU medido queda debajo de la recta U = X·D (la calibración sobrestimó D) y el colapso llega con 75.5 % de CPU, no con 100 %; justo ahí los hilos saltan de 12 a 43–55 y el RSS crece por la cola de peticiones. Los puntos que vuelven hacia menos usuarios en G6–G8 son del drenaje.

Prueba 2b (opcional): escalabilidad — el mismo estrés con 2 CPU

Evidencia: `resultados/escalabilidad/20261111-2103/` · 21:03:10–21:25:21 · `CPUS=2`, `TOP=2400` · código 99 (red de seguridad a los ~1,315 s). Corrida válida (V-01 k6 1.7 % por ventana; equipo 42.6 %; V-05 `cpu.max = 200000 100000`).

Indicador	1 CPU (`estres/20261111-1940`)	2 CPU (`escalabilidad/20261111-2103`)
Capacidad U-08	**850**	**1,300** (× 1.53)
Última ventana que cumple / primera falla sostenida	876 (min 11.8) / 946 (min 12.2)	1,337 (min 16.2) / 1,417 (min 16.8)
Throughput máximo atendido	85.2 req/s	129.7 req/s (× 1.52)
CPU de Qolqe al colapsar (techo)	75.5 % (de 100 %)	112.8 % (de 200 %)
CPU por sesión bajo carga = CPU ÷ (L ÷ W)	0.726 ÷ (822 ÷ 120) ≈ 106 ms	1.13 ÷ (1,337 ÷ 120) ≈ 101 ms
Throttling del cgroup	525 períodos (63.5 s)	217 períodos (26.6 s)
RSS / hilos / descriptores máximos	478 MB / 55 / 3,152	719 MB / 56 / 5,622

Indicador	1 CPU (`estres/20261111-1940`)	2 CPU (`escalabilidad/20261111-2103`)
Errores de rendimiento (toda la corrida)	116 respuestas 500 + 4,762 timeouts de 30 s	120 respuestas 500 + 9,960 timeouts
`database is locked` en el log de Qolqe	829 (733 en `autenticacion.py:136`, 93 en `mover_saldo`)	1,280 (1,202 en `autenticacion.py:136`, 78 en `mover_saldo`)
Corte de k6 (red de seguridad del 20 %)	952 s, 1,103 simultáneos	~1,315 s, 1,656 simultáneos
U-07 integridad del dinero	0 descuadres	0 descuadres

Estrés con 2 CPU, momentos seleccionados (`escalabilidad/20261111-2103/tabla_estres.md`; ms):

Mom.	Simult.	p95	p95 cons.	p95 dinero	Error %	req/s	CPU %	RSS MB	Hilos
T1	101	78.8	77.3	124	0.0	10.0	9.3	59.8	7.4
T3	326	85.0	80.3	119	0.0	32.6	35.8	62.7	8.4
T5	527	80.3	75.5	118	0.0	52.8	49.5	65.3	10.3
T7	726	77.5	73.2	116	0.0	70.3	64.4	67.4	10.7
T9	923	78.8	72.5	117	0.0	91.6	79.4	69.6	11.2
T11	1,128	74.4	71.8	120	0.0	108	95.8	74.6	12.8
T12	1,228	79.2	71.6	116	0.0	119	104	75.8	13.5
T13	1,337	81.2	72.1	119	0.0	130	113	76.2	13.2
T14	**1,417**	**18,307**	18,144	18,476	**23.0**	**17.6**	51.8	160	47.3
T15	**1,656**	**30,002**	30,002	30,002	**99.1**	**3.4**	51.5	339	48.9

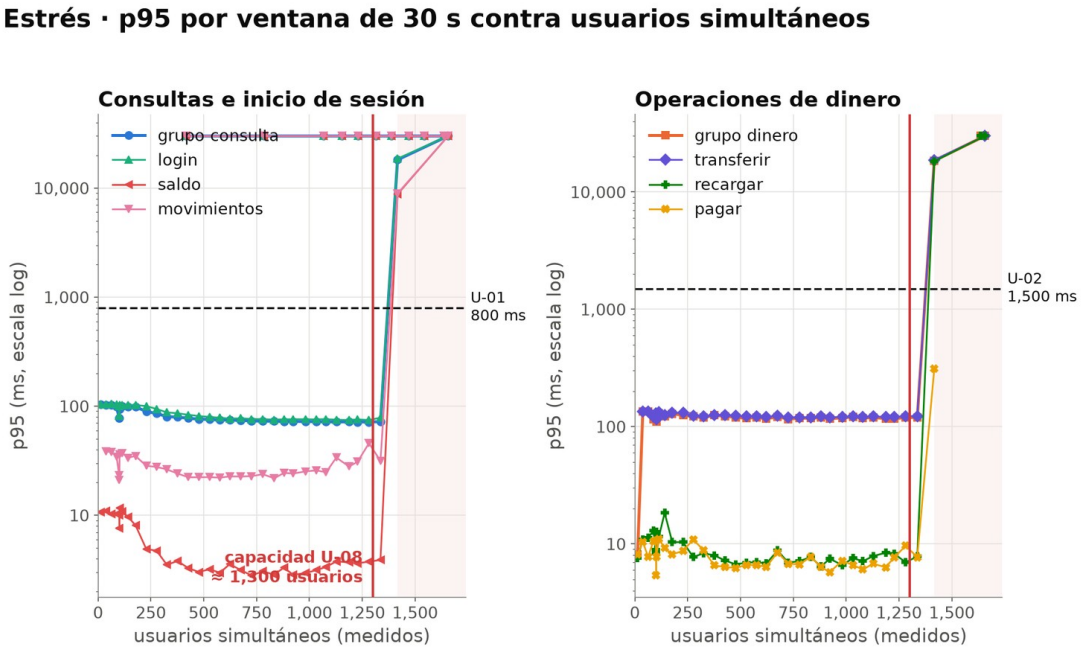


Figura 10. Estrés 2 CPU: p95 contra usuarios simultáneos; capacidad 1,300 (`escalabilidad/20261111-2103/graficos/G6_p95_vs_usuarios.png`).

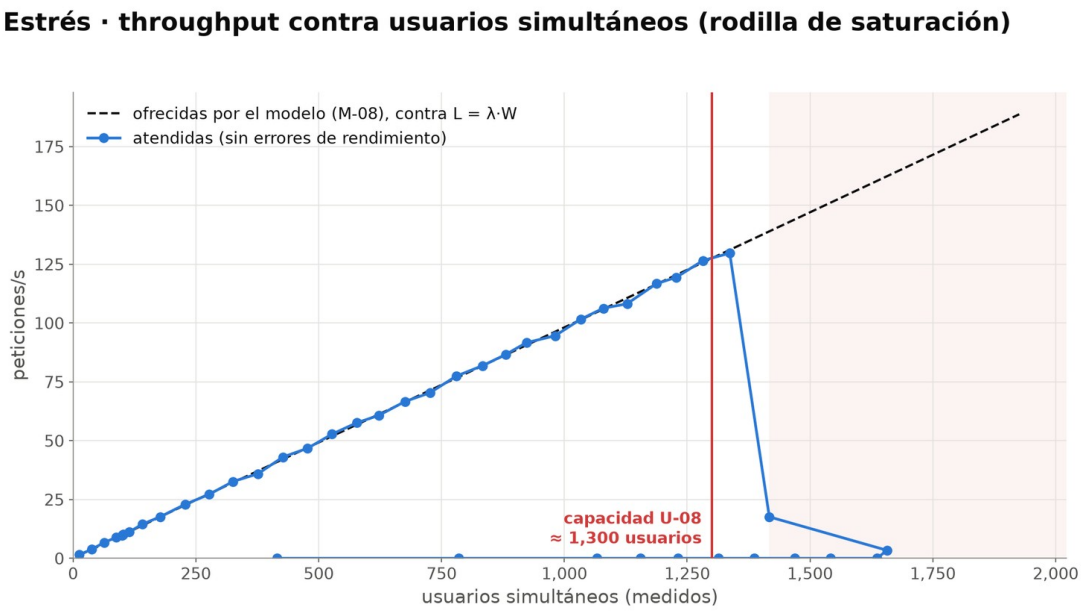


Figura 11. Estrés 2 CPU: throughput atendido contra ofrecido; rodilla en ~1,337 usuarios (`G7_throughput_vs_usuarios.png`).

Lectura: duplicar el CPU sube la capacidad × 1.53 y el throughput × 1.52, no × 2, y el colapso vuelve a ser brusco (81 ms → 18.3 s de p95 entre 1,337 y 1,417) con el CPU al 113 % de un techo de 200 %. Colapsar con CPU libre indica que el cuello no es solo el CPU (§8).

Prueba 3: Pico — campaña “Quincena Qolqe”, 100 → 300 → 100 usuarios simultáneos

Evidencia: `resultados/pico/20261111-1918/` · inicio 19:18:14 · fin 19:39:18 · código de salida de k6: **0**. **Corrida válida** (V-01 k6 0.3 % por ventana; V-02 0; V-03 0.48 pp; V-04 81.4 %; V-05 ok). Máximo de simultáneos medido: **304** (esperado 300). Es la repetición del pico `20261111-1852` (descartado, §6).

Umbral	Fase	Límite	Medido	¿Cumple?
U-01 / U-02 antes del pico	`base` (11:58–12:00)	p95 < 800 / 1,500	80 / 123	✓
U-06 consultas e inicio de sesión	campaña (12:00–12:15)	p95 < 1,600	grupo 78; login 97	✓ Cumple
U-06 dinero	campaña	p95 < 3,000	grupo 123; transferir 126	✓ Cumple
U-06 error y sesiones descartadas	campaña	≤ 1 %; 0	0.00 % (8,729 operaciones); 0	✓ Cumple
U-06 recuperación	`recuperacion` (12:10–12:15)	p95 < 800 / 1,500	80 / 123	✓ Cumple
Contra los umbrales normales (información para el negocio)	campaña	p95 < 800 / 1,500	78 / 123 (cada endpoint también)	✓ (informativo)
U-07 integridad del dinero	toda la corrida	0 descuadres	0 (BD = 201 recibidos: 1,296 transferencias, 868 recargas, 339 pagos)	✓ Cumple

Errores de rendimiento: 0. Errores de datos: 0. En la campaña: 14,585 peticiones a 16.2 req/s de promedio; la ventana más alta atendió **30.2 req/s** frente a 29.35 ofrecidos en el salto (`pico/20261111-1918/tabla_resumen.md` y `ventanas.csv`). p95 / p99 de la campaña: login 97 / 134, transferir 126 / 134, movimientos 34 / 55, saldo 8 / 12, recargar 14 / 21, pagar 13 / 19.

- **Recursos** (CPU y memoria de Qolqe y de la herramienta):

Proceso	Base	Salto (hasta 304 simultáneos)	Recuperación	Otros
Qolqe CPU (% de 1 CPU)	10.0 %	25.4 % (31.9 % por ventana; 51 % en 1 s)	10.3 %	throttling 18 períodos (0.8 s)
Qolqe RSS / hilos máx.	61.8 MB / 9	65.2 MB / 12	65.8 MB / 9	RSS máx. 66.4 MB
k6 CPU / memoria	—	0.3 % del equipo por ventana; 29 % de un núcleo en 1 s; 329 MB	—	V-01 < 80 %: ✓
Equipo completo	—	90.2 % en las ventanas de los min 10.25–11.25	—	informativo (macOS)

- **Gráficos** (tiempo de respuesta, errores, throughput y recursos a lo largo de la prueba):

Pico · Usuarios simultáneos medidos frente a la Ley de Little

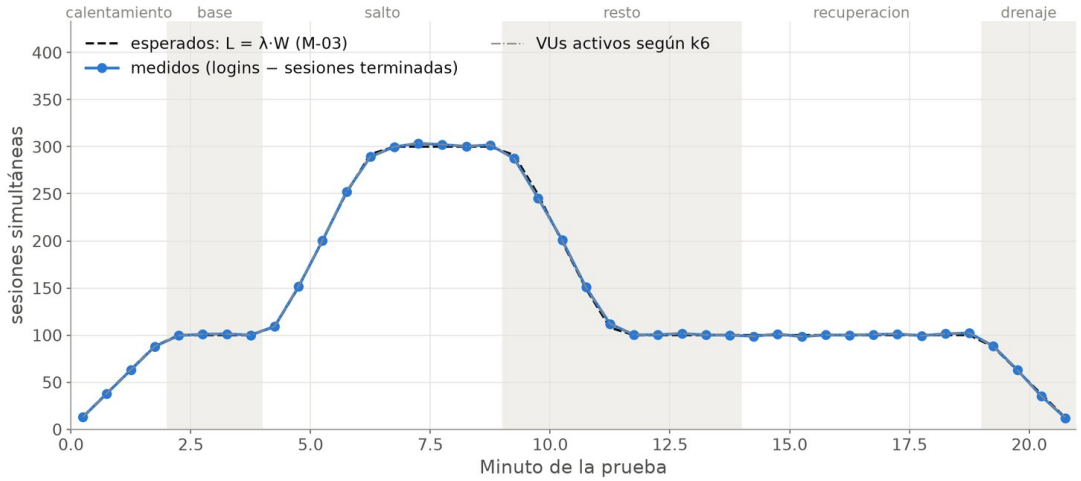


Figura 12. Pico: usuarios simultáneos medidos frente a la Ley de Little (100 → 304 → 100) (`pico/20261111-1918/graficos/G4_usuarios_simultaneos.png`).

Pico · Tiempo de respuesta por ventana de 30 s (p95 y p50)

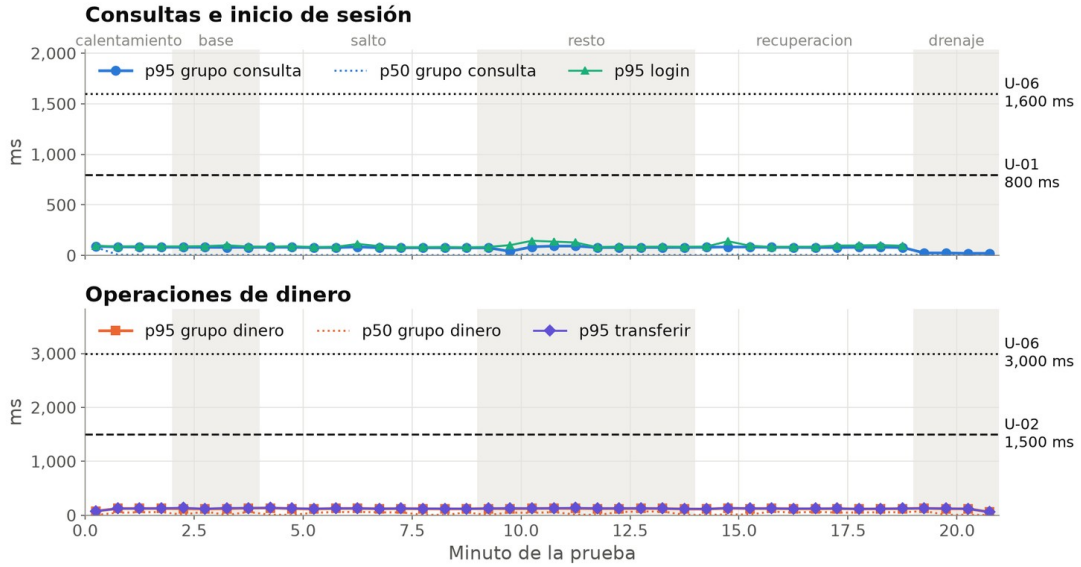


Figura 13. Pico: p95 y p50 por ventana de 30 s, con las fases y los umbrales normales y de campaña (`G1_tiempo_respuesta.png`).

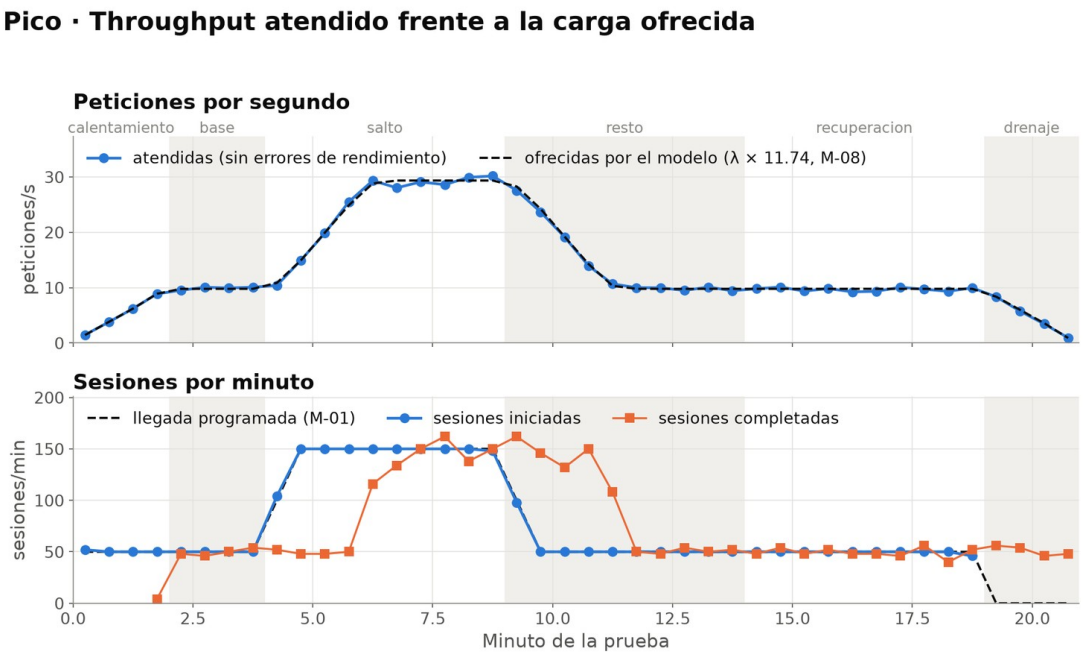


Figura 14. Pico: throughput atendido frente al ofrecido (`G2_throughput.png`).

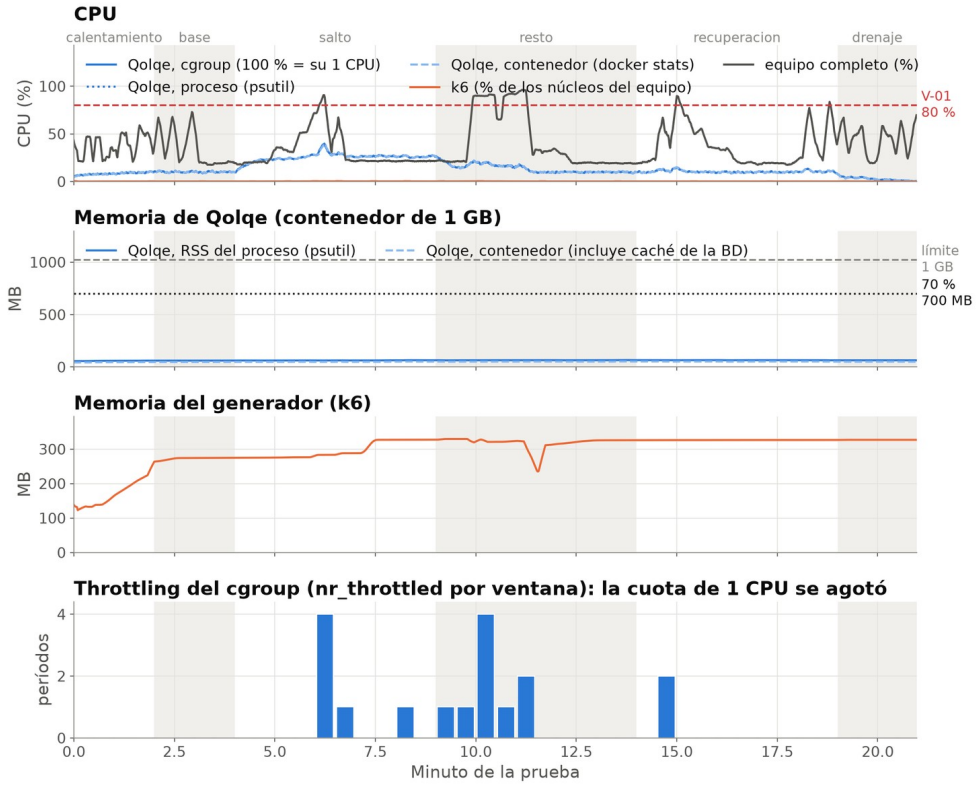


Figura 15. Pico: recursos de Qolqe y de k6 (`G5_recursos.png`).

Lectura: los simultáneos suben de 100 a 304 en ~2 min tras el salto y vuelven a ~100 en la recuperación, como predice Little. El p95 casi no cambia con el salto (campaña: consulta 78 ms, dinero 123 ms); la única subida visible es la del login, hasta ~145 ms en las ventanas de los minutos 10.25–11.25, cuando procesos de macOS llevaron el CPU del equipo al 90 %.

Qolqe usó en promedio el 25 % de su CPU en el salto. `G3_errores.png` muestra 0 errores en toda la corrida.

Prueba 4: Resistencia — 60 min a 100 usuarios simultáneos (3,000 sesiones), proyección a 18 h

Evidencia: `resultados/resistencia/20261111-1958/` · inicio 19:58:20 · fin 21:02:32 · código de salida de k6: **0**. **Corrida válida** (V-01 k6 0.2 % por ventana; equipo 26.6 %; V-03 0.11 pp; V-04 69.4 % ocioso con carga de 1 min 12.01; V-07 0 avisos).

Umbral	Límite	Medido	¿Cumple?
U-01 / U-02 (60 min)	p95 < 800 / 1,500	95 (login 106) / 127 (transferir 130)	✓ Cumple
U-03 / U-04 / U-05	≤ 1 %; ≥ 9.6 req/s y ≥ 49/min; 0 descartadas, 0 ventanas, 90–110	0.00 % (20,997 operaciones); 9.77 req/s y 50.0/min; 0; 0 de 60; 97.2–102.4	✓ Cumple
U-09(a) ventanas de 5 min que cumplen U-01 a U-03	12 de 12	12 de 12	✓ Cumple
U-09(b) RSS proyectado a 54,000 sesiones	< 700 MB	**92 MB** (pendiente 0.56 MB por 1,000 sesiones, r^2 0.53)	✓ Cumple
U-09(c) hilos y descriptores proyectados	≤ 2 × el inicial	hilos 5.8 (límite 15.3); descriptores 43.1 (límite 85.7)	✓ Cumple
U-09(d) deriva del p95: últimos 10 min ÷ primeros 10	≤ 1.2 ×	consulta 98 → 96 (0.98 ×); dinero 126 → 127 (1.01 ×)	✓ Cumple
U-07 integridad del dinero	0 descuadres	0 (BD = 201 recibidos: 2,795 transferencias, 1,860 recargas, 737 pagos)	✓ Cumple

Errores de rendimiento: 0; de datos: 0; mezcla a ≤ 0.11 pp de la pedida. p95 por endpoint en el estable: login 106, transferir 130, movimientos 39, saldo 11, recargar 12, pagar 11 (`tabla_resumen.md`).

- **Recursos** (CPU y memoria de Qolqe y de la herramienta):

Proceso	CPU promedio	CPU máximo	Memoria	Otros
Qolqe (% de 1 CPU)	12.1 % en el estable (11.8 % toda la corrida)	12.6 % por ventana de 5 min; 37.7 % en 1 s	RSS 60.6 → 63.1 MB (ventanas estables); máx. 63.3	hilos 7.7 → 7.3; descriptores 42.8 → 42.7; throttling 207 períodos (3.1 s); log `qolqe.log.gz` 170 KB
k6 (% del equipo)	0.21 %	0.24 % por ventana; 1.6 % en 1 s (16 % de un núcleo)	153 MB máx.	V-01: ✓
Equipo completo	17.4 %	26.6 % por ventana	—	informativo

- **Gráficos** (tiempo de respuesta, errores, throughput y recursos a lo largo de la prueba):

Resistencia · Tiempo de respuesta por ventana de 5 min (p95 y p50)

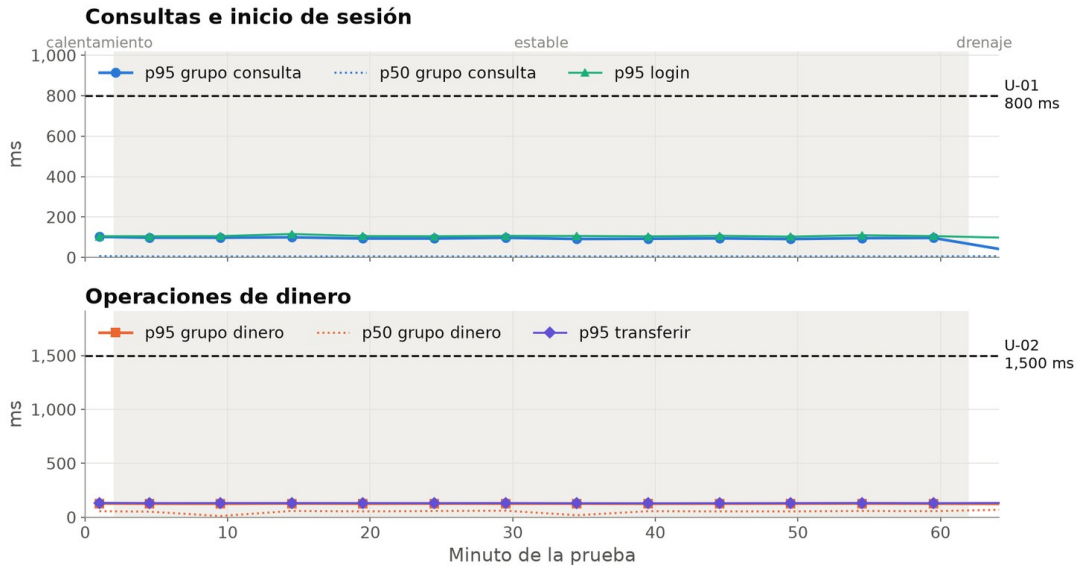


Figura 16. Resistencia: p95 y p50 por ventana de 5 min durante 60 min (`resistencia/20261111-1958/graficos/G1_tiempo_respuesta.png`).

Resistencia · memoria contra sesiones acumuladas y proyección a 54,000 (U-09b)

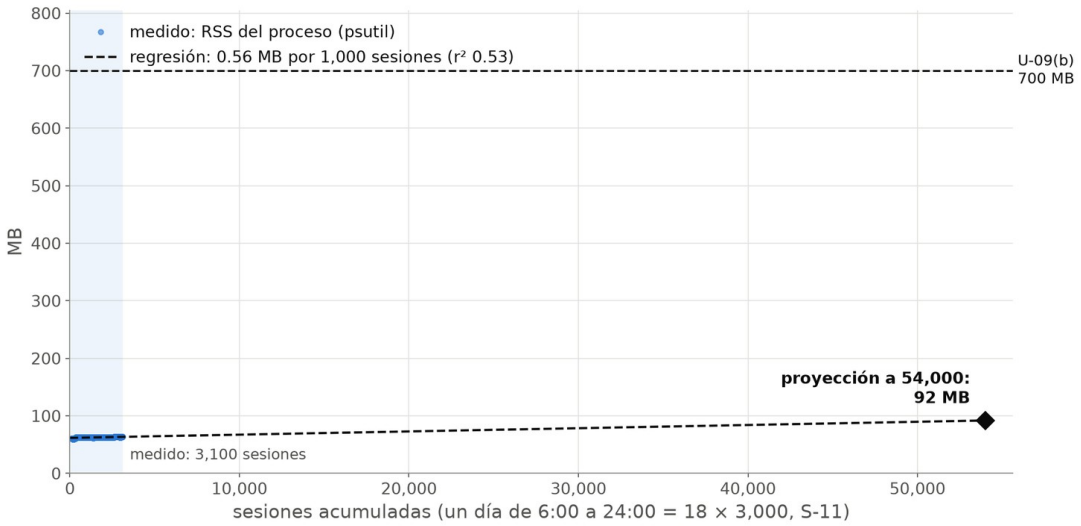


Figura 17. Resistencia: RSS contra sesiones acumuladas, regresión y proyección a 54,000 sesiones (`G9_memoria_vs_sesiones.png`).

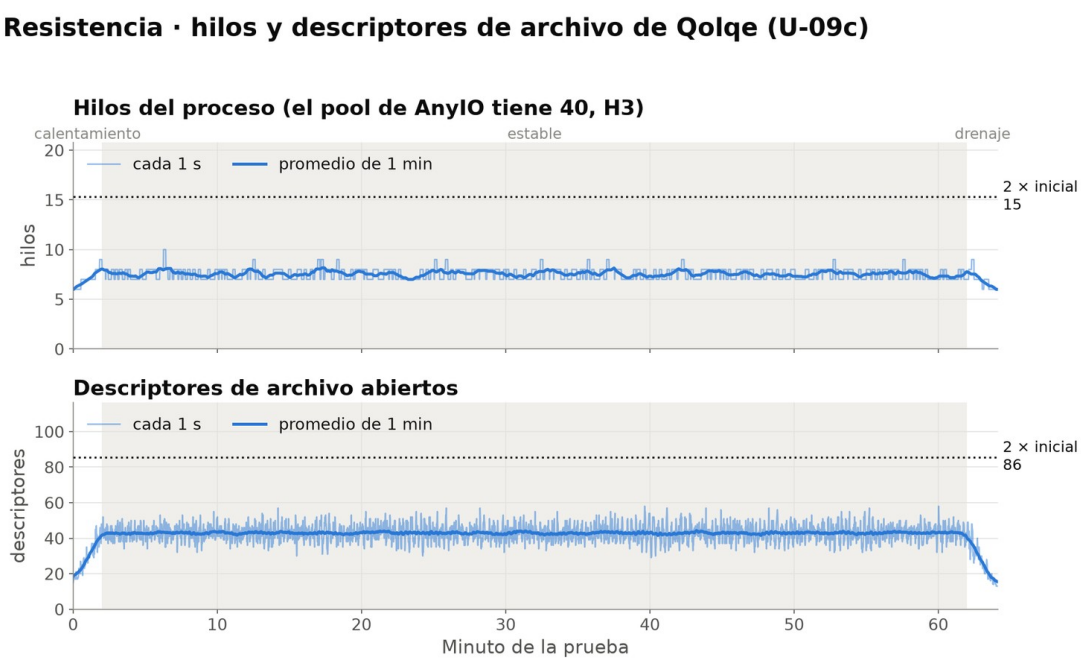


Figura 18. Resistencia: hilos y descriptores a lo largo de la prueba (`G10_hilos_descriptores.png`).

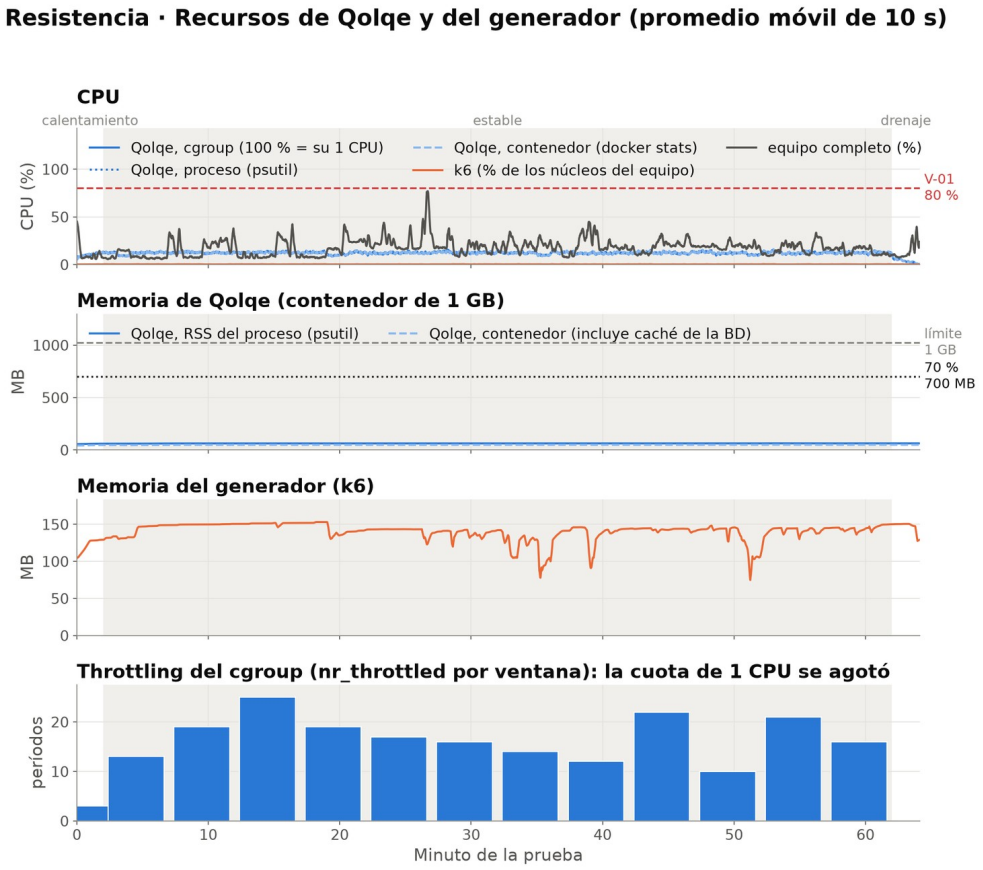


Figura 19. Resistencia: recursos de Qolqe y de k6 (`G5_recursos.png`).

Lectura: el p95 queda plano los 60 min (sin deriva), el RSS casi no crece (60.6 → 63.1 MB por ventana de 5 min) y la proyección a un día completo da 92 MB, lejos de 700 MB; hilos y descriptores no tienen tendencia. No hay señales de fuga en 3,101 sesiones; la proyección a 18 h es una extrapolación lineal (r^2 0.53) y se declara como tal. `G2_throughput.png` muestra 9.77 req/s sostenidos.

Ensayo de la demostración en vivo y comparativo de los escenarios

Demo (`resultados/demo/20261111-2125`, 21:26:03–21:28:32, ≈ 2.5 min con dashboard de k6 en `localhost:5665`). Tiene dos partes: hora punta a escala y un tramo **sin think time** a propósito (S13: “una prueba de estrés disfrazada”). En hora punta cumple (consultas 103 ms, dinero 138 ms, 0.00 % de error). Sin pausa falla como se espera: p95 de consultas 15,689 ms, login 17,886 ms, dinero 11,713 ms y 35.07 % de operaciones fallidas (101 respuestas 500 y 81 errores de conexión al cortar); k6 termina con código 99. U-07: 0 descuadres (10 transferencias y 6 recargas cortadas en vuelo quedaron aplicadas y concilian). En clase el dashboard mostrará errores en rojo, no solo lentitud.

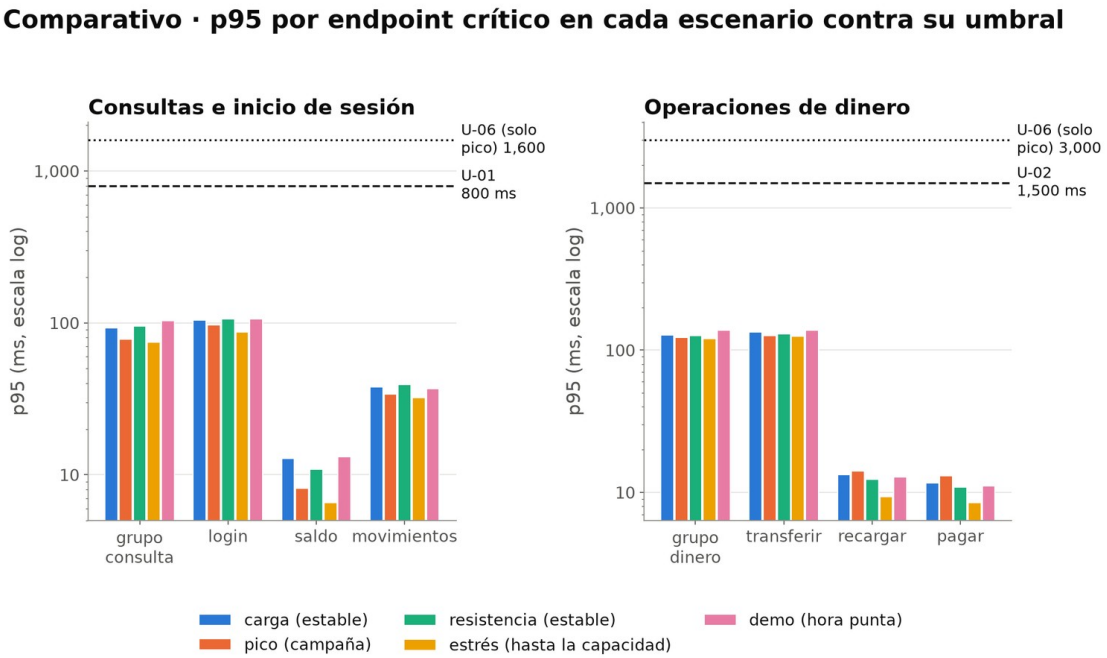


Figura 20. Comparativo: p95 de cada endpoint crítico en cada escenario contra su umbral (`resultados/comparativo/G11_comparativo.png`).

Veredicto por umbral y escenario (`comparativo/veredictos.md`; ✓ cumple, ✗ falla, — no aplica):

Umbral	Ref.	Calib.	Carga	Pico	Estrés 1 CPU	Estrés 2 CPU	Resist.	Demo*
U-01 consultas e inicio de sesión	—	—	✓	—	—	—	✓	✓
U-02 dinero	—	—	✓	—	—	—	✓	✓
U-03 error ≤ 1 %	—	—	✓	—	—	—	✓	✓
U-04 volumen de la hora punta	—	—	✓	—	—	—	✓	—
U-05 sin acumular retrasos	—	—	✓	—	—	—	✓	—
U-06 campaña	—	—	—	✓	—	—	—	—
U-07 dinero íntegro	✓	✓	✓	✓	✓	✓	✓	✓
U-08 capacidad	—	—	—	—	850	1,300	—	—
U-09 sin reinicios	—	—	—	—	—	—	✓	—
V-06 1 VU ≤ 10 % del umbral	✗ login	—	—	—	—	—	—	—

** Demo: solo su parte de hora punta. El estrés no tiene veredicto de U-01 a U-05 porque mide la capacidad.*

Resumen por escenario (`comparativo/resumen_escenarios.csv`):

Escenario	Peticiones	p95 consulta (ms)	p95 dinero (ms)	Error rend. %	req/s máx.	Simult. máx.	CPU Qolqe máx. (ventana)	RSS máx. MB
carga (estable)	8,796	93	128	0.00	10.2	101	13.3	63
pico (campaña)	14,585	78	123	0.00	30.2	304	31.9	66
estrés 1 CPU (hasta la capacidad)	25,589	75	121	0.00	85.2	1,144*	75.5	478
estrés 2 CPU (hasta la capacidad)	55,536	73	119	0.00	129.7	1,656	112.8	719
resistencia (estable)	35,182	95	127	0.00	9.8	101	12.6	63
demo (hora punta)	430	103	138	0.00	23.5	134	63.4	110

** El máximo de simultáneos del estrés 1 CPU incluye las ventanas del colapso y del drenaje.*

---

8. Análisis, conclusiones y recomendaciones

- ¿Se cumplieron los umbrales? ¿Dónde y con cuántos usuarios dejaron de cumplirse?
  - **Hora punta (carga, 100 simultáneos): cumple todo** con mucho margen: p95 de consultas 93 ms (login 105) frente a 800 ms, de dinero 128 ms (transferir 134) frente a 1,500 ms, 0.00 % de error, 9.77 req/s y 0 sesiones descartadas.
  - **Pico de la campaña (304 simultáneos): cumple U-06 e incluso los umbrales normales** (78 / 123 ms), con 0.00 % de error, 0 sesiones descartadas y recuperación en 12:10–12:15 (80 / 123 ms).
  - **Operación continua (60 min): cumple U-09**; sin deriva del p95 (0.98 × y 1.01 ×) ni crecimiento de memoria (proyección 92 MB).
  - **Integridad del dinero: 0 descuadres en las 8 corridas analizadas** (y en el pico descartado), incluso en el colapso: ninguna operación quedó a medias ni se aplicó dos veces.
  - **Dónde dejaron de cumplirse (estrés):** con 1 CPU, entre 876 simultáneos (última ventana que cumple, minuto 11.8) y **946** (minuto 12.2), todos los criterios a la vez: p95 de 85 ms a 13.4 s, error 13.7 %, throughput de 85.2 a 26.1 req/s. Con 2 CPU, entre 1,337 y 1,417. La hipótesis previa (1,500 ± 300) queda **refutada**.
- Patrón de degradación y **cuello de botella**, con evidencia:

Método (S14): ¿pasó? → ¿dónde falló? → ¿por qué? → ¿qué hacer? Las hipótesis se prepararon antes de correr, cada una con lo que se vería si fuera el cuello:

H	Cuello candidato	Qué se vería si es el cuello	Resultado y evidencia
H1	**CPU por el PBKDF2 del login** (`credenciales.py:13,24`)	Login primero en fallar; CPU ≈ 100 % y throttling; py-spy en `verificar_pin`; patrón **gradual**; rodilla cerca de L_sat	**Confirmada como el mayor consumo, no como disparador.** `verificar_pin` = 53 % de las muestras en la base y 63 % en el colapso con 1 CPU (59 % y 47 % con 2 CPU). Pero el colapso llega con CPU al 75.5 %, es abrupto y con 2 CPU la capacidad crece × 1.53, no × 2
H2	**CPU por el N+1 de movimientos** (`movimientos.py:113-123`)	`movimientos` con la mayor razón p95 ÷ p95 de referencia; py-spy en `listar`	**Secundaria.** ≈ 6 % de las muestras en la base (línea 118); ≈ 32 % en la saturación con 2 CPU (líneas 113, 118 y 122): gana peso cuando sobra CPU. Su p95 no falla antes que el resto

H	Cuello candidato	Qué se vería si es el cuello	Resultado y evidencia
H3	**Pool de 40 hilos** (el antifraude duerme 40–120 ms, `operaciones.py:101`)	Hilos ≥ 40; p95 sube con CPU < 80 %; patrón **abrupto**; hilos en espera en la foto	**Confirmada como amplificador.** Hilos de 12 a 43 –55 en el colapso (T9–T10); con 2 CPU las 40 plazas del pool de AnyIO están en `iniciar_sesion` línea 136 esperando a SQLite, no en el `sleep` del antifraude
H4	**SQLite**: un solo escritor, `busy_timeout` de 10 s (`bd.py:209,212`)	`500 database is locked`; POST antes que GET; py-spy en `execute`/`commit`	**Confirmada como disparador.** 116 y 120 respuestas 500; en el log, 733 de 829 y 1,202 de 1,280 excepciones salen del INSERT de sesión del login (`autenticacion.py:136`); py-spy: 19.6 % de las muestras en esa línea (1 CPU) y 6.5 % (2 CPU)
H5	**Memoria**	RSS que crece sin parar; p95 que empeora con el tiempo	**Descartada.** Resistencia: RSS 60.6 → 63.1 MB, proyección 92 MB < 700. El RSS solo crece en el colapso (478 / 719 MB) por la cola
H6	**Generador o entorno**	CPU de k6 > 80 % o errores de conexión con Qolqe ocioso	**Descartada.** k6 ≤ 1.7 % del equipo por ventana; las ventanas del equipo > 80 % no coinciden con el colapso

Patrón de degradación: abrupto (un precipicio), no gradual. Con 1 CPU, en 822 simultáneos el p95 era 85 ms con el CPU de Qolqe al 72.6 %; en la ventana siguiente (946) subió a 13.4 s con 13.7 % de error y el CPU **bajó** al 60 %; con 1,103 hubo 92.5 % de error. Primero aparecen respuestas 500 y luego timeouts de 30 s (Figura 8). Es la firma de un recurso que se agota de golpe (“el pool de 50 conexiones se llenó”, S14), no la de un CPU que se satura de a poco.

Evidencia 1 — dónde se va el CPU (py-spy). Porcentaje de las muestras del perfil (`perfil_base.svg`, `perfil_saturacion.svg` de cada corrida):

Perfil	`verificar_pin` (PBKDF2)	`autenticacion.py:136` (INSERT de sesión)	`movimientos.py` (N+1)	`mover_saldo`
Base, 100 usuarios, 1 CPU (617 muestras)	53.2 %	0.2 %	≈ 7 % (118: 5.7 %)	—
Colapso, 790–850 s, 1 CPU (2,519 muestras)	63.3 %	**19.6 %**	≈ 2.5 %	2.9 %
Base, 100 usuarios, 2 CPU (734 muestras)	59.4 %	—	≈ 7 % (118: 6.1 %)	0.3 %
Inicio del colapso, 1,000–1,060 s, 2 CPU (3,791 muestras)	46.6 %	**6.5 %**	**≈ 32 %** (113: 10.6 %; 118: 14.6 %; 122: 7.2 %)	1.0 %

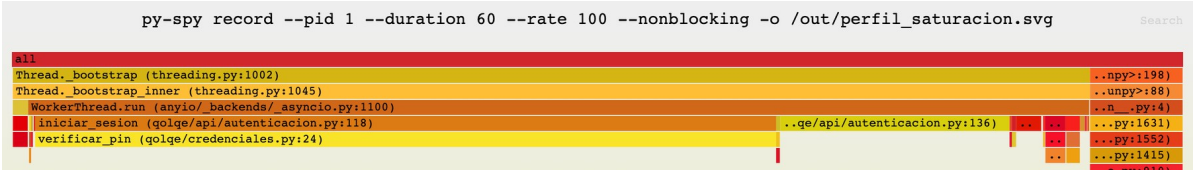


Figura 21. py-spy con 1 CPU entre los 790 y 850 s (ya en el colapso): `verificar_pin` ocupa el 63 % del ancho y `autenticacion.py:136`, el bloque siguiente, el 19.6 % (`estres/20261111-1940/perfil_saturacion.svg`, recortado a los primeros niveles).

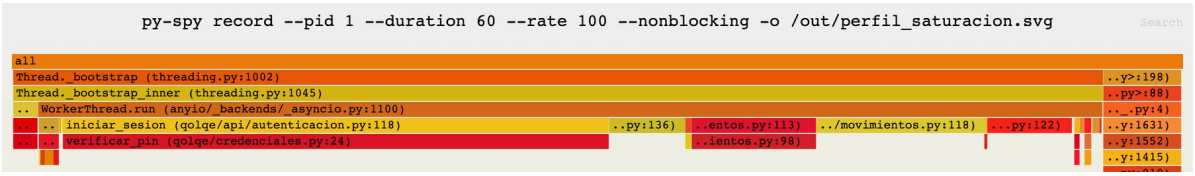


Figura 22. py-spy con 2 CPU entre los 1,000 y 1,060 s: `verificar_pin` 46.6 %, línea 136 6.5 % y las líneas 113, 118 y 122 de movimientos ≈ 32 % (`escalabilidad/20261111-2103/perfil_saturacion.svg`).

Evidencia 2 — en qué están los hilos al colapsar. Foto de hilos con 2 CPU a los 1,250 s (`escalabilidad/20261111-2103/hilos_meseta.txt`): 45 hilos; **40 hilos de trabajo de AnyIO (todas las plazas de su límite de 40) detenidos en la misma línea**, 4 hilos ociosos sin plaza y el hilo principal:

```
Thread 91 (idle): "AnyIO worker thread"
    iniciar_sesion (qolqe/api/autenticacion.py:136)
    WorkerThread.run (anyio/_backends/_asyncio.py:1100)
… (los mismos 3 marcos en 40 hilos)
autenticacion.py:135-137:
    sesion_id = bd.execute("INSERT INTO sesiones (jti, usuario_id, dispositivo_id,
…) VALUES (?, ?, ?, ?, ?)", …)
```

Evidencia 3 — `database is locked`. Las 116 respuestas 500 (1 CPU) y 120 (2 CPU) son `sqlite3.OperationalError: database is locked`. En `qolqe.log.gz` hay 829 excepciones (1 CPU): **733** en el INSERT de sesión del login (`autenticacion.py:136`), 93 en `mover_saldo` (recargas 59, transferencias 34) y 3 en pagos; con 2 CPU, 1,202 de 1,280 en el login y 78 en `mover_saldo`. Las excepciones son más que las respuestas 500; la explicación más probable es que muchas ocurrieron en peticiones que k6 ya había dado por perdidas a los 30 s (se contaron como timeout). Mecanismo: SQLite admite **un solo escritor**; cada login escribe una fila en `sesiones` y espera el bloqueo hasta 10 s (`busy_timeout`, `bd.py:212`). Con la ráfaga de logins, los escritores hacen cola, cada uno ocupa un hilo del pool mientras espera, el pool de 40 se llena y **todas** las peticiones, también las GET, quedan en cola hasta el timeout del cliente.

```
File "/app/qolqe/api/autenticacion.py", line 136, in iniciar_sesion
sqlite3.OperationalError: database is locked
```

Evidencia 4 — la ley de utilización (aporte propio: si cada sesión gasta D segundos de CPU, 1 CPU alcanza para 1 ÷ D sesiones por segundo y la saturación llega en L_sat = W ÷ D). Con D = 157 ms de la calibración, L_sat = 763; bajo carga D fue ≈ 106 ms (0.726 ÷ (822 ÷ 120)), lo que daría L_sat ≈ 120 ÷ 0.106 ≈ 1,130 al 100 % de CPU (Figura 7). El colapso llegó antes, en 946 y con 60–75 % de CPU. Con 2 CPU pasó lo mismo (113 % de 200 %). Si el único cuello fuera el CPU, la capacidad se habría duplicado.

Conclusión: el cuello de botella es la escritura de la sesión en SQLite durante el login (H4), que llena el pool de 40 hilos (H3), sobre un login que ya consume la mayor parte del CPU por el PBKDF2 (H1). El PBKDF2 hace lento cada login y alarga el tiempo en que los escritores compiten; el bloqueo de SQLite es lo que convierte esa competencia en un colapso brusco. H2 es secundaria (aparece cuando sobra CPU); H5 y H6 quedan descartadas.

- Capacidad estimada (usuarios simultáneos que cumplen los criterios):

850 usuarios simultáneos con 1 CPU y 1 GB (≈ **25,500 sesiones/h**): **8.5 ×** la hora punta (100) y **2.8 ×** el pico de la campaña (300). Con 2 CPU, **1,300** (≈ 39,000 sesiones/h; 13 × y 4.3 ×). Es una **línea base relativa** de un contenedor en una laptop, no la capacidad de producción (§6). La capacidad sostenida no se confirmó con una corrida a carga constante de 850 (`L_CAP`, opcional no ejecutado).

- Riesgos para el negocio:
  - **Colapso sin aviso:** entre 876 y 946 simultáneos el p95 pasa de ~80 ms a 13 s y el error de 0 % a 14 %. No hay degradación gradual que sirva de alarma: si la campaña crece más de lo previsto, el servicio pasa de “bien” a “caído” en un minuto.
  - **El login es el punto único de falla:** la ráfaga de la campaña (“la mayoría inicia sesión en los primeros 5 minutos”) cae justo sobre la operación más cara y la que escribe en SQLite. Si el login se bloquea, el usuario no entra y no puede operar.

    - **Operaciones aplicadas sin confirmación:** en el colapso hubo transferencias y recargas aplicadas cuyo cliente recibió un timeout (2 transferencias con 1 CPU; 1 transferencia y 1 recarga con 2 CPU). k6 no reintenta, así que no hubo duplicados; una app real que reintente sí podría cobrar dos veces (no hay clave de idempotencia).
    - **Carrera en transferencias simultáneas del mismo usuario (DEF-21):** la prueba no la ejercita (un usuario por VU), pero un usuario con dos dispositivos sí; la verificación C5 la detectaría.
    - **Margen relativo:** el 2.8 × sobre la campaña se midió en una laptop; en producción puede ser mayor o menor. Lo que sí se traslada es el patrón: el límite lo pone SQLite antes que el CPU.
- Acciones recomendadas:

Son recomendaciones, en orden de impacto (`qolqe/` no se modifica en este proyecto):

- **Sacar la escritura de la sesión del camino crítico del login (H4):** transacción más corta o registro de sesión fuera del bloqueo de escritura; para varios escritores, una base de datos de servidor (PostgreSQL) o varios procesos con la sesión en un almacén aparte.
- **Fallar rápido en lugar de encolar (H3):** bajar el `busy_timeout` y limitar la concurrencia de entrada con respuesta 503 + `Retry-After`, para que un pico produzca rechazos cortos y no timeouts de 30 s que arrastran a las GET. Ampliar el pool solo junto con lo anterior.
- **Reducir cuántas veces se paga el PBKDF2 (H1):** no bajar las iteraciones (OWASP recomienda subirlas), sino renovar el token para no iniciar sesión en cada visita, y escalar CPU (con 2 CPU la capacidad subió × 1.53).
- **Corregir el N+1 de movimientos (H2):** saldo acumulado con `SUM() OVER (…)` y paginación en SQL; pesa el 32 % del CPU cuando sobra CPU.
- **Clave de idempotencia** en transferencias, recargas y pagos, para que un reintento tras un timeout no duplique el cobro.
- **Para la campaña:** sala de espera virtual o escalonar los logins de las 12:00, y alertas sobre el p95 del login y los 500 `database is locked`, que aparecen un minuto antes del colapso.
- **Volver a medir** tras cada cambio con el mismo harness (`correr.sh estres`), confirmar la capacidad sostenida (`L_CAP=850`) y correr la resistencia completa de 18 h en un entorno dedicado. El SLA no necesita renegociarse: hora punta y campaña cumplen con margen.